BỘ TÀI CHÍNH

TRƯỜNG ĐẠI HỌC TÀI CHÍNH – MARKETING

KHOA KHOA HỌC DỮ LIỆU

ĐỒ ÁN MÔN HỌC

KIỂM THỬ PHẦN MỀM

Tên đề tài:

KIỂM THỬ WEBSITE AUTOMATION TEST STORE

Sinh viên thực hiện: Lê Thị Thanh Hiền

Mã số sinh viên: 2221004176

Tên lớp học phần: Kiểm thử phần mềm

Mã lớp học phần: 2521101165001

TP. HCM, ngày 12 tháng 08 năm 2025

BỘ TÀI CHÍNH

TRƯỜNG ĐẠI HỌC TÀI CHÍNH – MARKETING

KHOA HỌC DỮ LIỆU

ĐỒ ÁN MÔN HỌC

KIỂM THỬ PHẦN MỀM

Tên đề tài:

KIỂM THỬ WEBSITE AUTOMATION TEST STORE

Giảng viên hướng dẫn: Trần Trọng Hiếu

Sinh viên thực hiện: Lê Thị Thanh Hiền

Mã số sinh viên: 2221004176

Tên lớp học phần: Kiểm thử phần mềm

Mã lớp học phần: 2521101165001

TP. HCM, ngày 12 tháng 08 năm 2025

turnitin

Biên nhận Điện tử

Biên nhận này xác nhận Turnitin đã nhận được bài của bạn. Dưới đây bạn sẽ thấy các thông tin trên biên nhận liên quan đến bài nộp của bạn.

Trang đầu bài nộp của bạn hiển thị bên dưới đây.

Tác giả nộp:	Hiền Lê Thị Thanh
Tựa đề bài tập:	25.2S3 NỘP TIỂU LUẬN KTPM TỪ 01/07/2025 ĐẾN 16/08/2025
Tựa bài nộp:	2221004176_LETHITHANHHIEN_TIEULUAN.docx
Tên Tập tin:	2221004176_LETHITHANHHIEN_TIEULUAN.docx
Kích cỡ tập tin:	857.75K
Đếm trang:	78
Đếm từ:	19,239
Đếm ký tự:	74,925
Ngày Nộp:	12-thg 8-2025 01:03CH (UTC+0700)
ID Bài Nộp:	2726564798

BỘ TÀI CHÍNH
TRƯỜNG ĐẠI HỌC TÀI CHÍNH – MARKETING
KHOA KHOA HỌC DỮ LIỆU

ĐỒ ÁN MÔN HỌC
KIỂM THỬ PHẦN MỀM

Tên đề tài:

KIỂM THỬ WEBSITE AUTOMATION TEST STORE

Sinh viên thực hiện:	Lê Thị Thanh Hiền
Mã số sinh viên:	2221004176
Tên lớp học phần:	Kiểm thử phần mềm
Mã lớp học phần:	2521101145001

TP. HCM, ngày 12 tháng 08 năm 2025

Bản Quyền 2025 Turnitin. Giữ Toàn Quyền.

LỜI CẢM ƠN

Trước tiên, em xin gửi lời cảm ơn đến các thầy cô trong Khoa Khoa Học Dữ Liệu đã xây dựng chương trình học hiệu quả, đặc biệt là đưa môn ***Kiểm thử phần mềm*** vào giảng dạy. Nhờ đó, em có cơ hội tiếp cận những kiến thức thiết thực, làm nền tảng cho việc học chuyên ngành cũng như công việc sau này.

Em xin gửi lời cảm ơn chân thành đến thầy ***Trần Trọng Hiếu*** – người đã tận tình hướng dẫn, hỗ trợ và chia sẻ nhiều kiến thức, kinh nghiệm trong suốt quá trình học và thực hiện đồ án. Nhờ sự chỉ dẫn của thầy, em đã có thể hoàn thành tốt bài làm của mình.

Do kiến thức và kinh nghiệm còn hạn chế, đồ án chắc chắn vẫn còn những thiếu sót. Em mong nhận được góp ý từ thầy để rút kinh nghiệm và cải thiện tốt hơn trong tương lai.

Em xin chân thành cảm ơn.

NHẬN XÉT CỦA GIẢNG VIÊN

Nhận xét của giảng viên hướng dẫn (chấm 1)	Nhận xét của giảng viên phản biện (chấm 2)
.... **Điểm số:** ..... **Điểm chữ**..... **Ngày……./……../202…..** **Ký tên (ghi rõ họ tên)**	.... **Điểm số:** ..... **Điểm chữ**..... **Ngày……./……../202…..** **Ký tên (ghi rõ họ tên)**

TP.HCM, ngày 21 tháng 04 năm 2022

MỤC LỤC

LỜI CẢM ƠN ii
NHẬN XÉT CỦA GIẢNG VIÊN iii
MỤC LỤC iv
DANH MỤC HÌNH ẢNH vi
DANH MỤC BẢNG vii
DANH MỤC THUẬT NGỮ ANH – VIỆT viii
DANH MỤC TỪ VIẾT TẮT ix
LỜI MỞ ĐẦU x
CHƯƠNG 1: TỔNG QUAN 1
1.1. Khái niệm 1
1.1.1. Các khái niệm liên quan 1
1.1.2. Các khái niệm kiểm thử 6
1.1.3. Kỹ thuật TestCase 16
1.1.4. Lỗi phần mềm 23
1.2. Mục tiêu nghiên cứu 34
1.3. Phạm vi nghiên cứu 35
1.4. Phương pháp nghiên cứu 35
1.4.1. Phương pháp định tính 35
1.4.2. Phương pháp định lượng 35
1.4.3. Phương pháp phân tích 36
1.4.4. Phương pháp thử nghiệm (Testing) 36
1.4.5. Phương pháp đối chiếu – so sánh 36
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT 37
2.1. Giới thiệu website 37
2.2. Giới thiệu quy trình hoạt động 38
2.2.1. Quản lý tài khoản 38
2.2.2. Quản lý danh mục sản phẩm và sản phẩm 39
2.2.3. Quản lý mua sản phẩm 41

CHƯƠNG 3: KIỂM THỬ PHẦN MỀM 43
3.1. Danh sách Testcase 43
3.1.1. Quản lý tài khoản 43
3.1.2. Quản lý mua sản phẩm Haircare 51
3.2. Danh sách Bug Report 57
CHƯƠNG 4: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 64
4.1. Kết quả đạt được 64
4.2. Hạn chế và đề xuất cải thiện 64
4.2.1. Hạn chế 64
4.2.2. Đề xuất 64
4.3. Hướng phát triển tiếp theo 65
TÀI LIỆU THAM KHẢO 1

DANH MỤC HÌNH ẢNH

Hình 3. 1: Dữ liệu đầu vào của Bug ID RE_05 58

Hình 3. 2: Bug screen của Bug ID RE_05 58

Hình 3. 3: Các sản phẩm trong danh mục Shampoo 60

Hình 3. 4: Các sản phẩm trong danh mục Conditioner 60

Hình 3. 5: Bug screen của Bug ID SP_01 60

Hình 3. 6: Dữ liệu đầu vào của Bug ID RV_01 61

Hình 3. 7: Bug screen của Bug ID RV_01 62

Hình 3. 8: Bug screen của Bug ID AC_01 63

DANH MỤC BẢNG

Bảng 3. 1: Danh sách Testcase chức năng Register 43

Bảng 3. 2: Danh sách Testcase chức năng Login 50

Bảng 3. 3: Danh sách Testcase chức năng Show products.....51

Bảng 3. 4: Danh sách Testcase chức năng Review 53

Bảng 3. 5: Danh sách Testcase chức năng Add to cart 54

Bảng 3. 6: Danh sách Testcase Checkout.....56

Bảng 3. 7: Bug report chức năng Register 57

Bảng 3. 8: Bug report chức năng Show products.....58

Bảng 3. 9: Bug report chức năng Review.....60

Bảng 3. 10: Bug report chức năng Add to cart.....62

DANH MỤC THUẬT NGỮ ANH – VIỆT

Tiếng Anh	Tiếng Việt
Software testing	Kiểm thử phần mềm
Manual testing	Kiểm thử thủ công
Automation testing	Kiểm thử tự động
Test plan	Kế hoạch kiểm thử
Test case	Trường hợp kiểm thử
Defect	Lỗi
Failure	Hỏng

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tên đầy đủ
IEEE	Institute of Electrical and Electronics Engineers
ISO	International Organization for Standardization
SRS	Software Requirements Specification
QA	Quality Assurance
UAT	User Acceptance Testing

LỜI MỞ ĐẦU

Trong bối cảnh công nghệ thông tin phát triển mạnh mẽ, phần mềm đã trở thành một yếu tố không thể thiếu trong hầu hết các lĩnh vực từ kinh doanh, giáo dục, y tế cho đến đời sống hằng ngày. Tuy nhiên, chất lượng phần mềm luôn là một thách thức lớn, bởi các lỗi (bug) có thể gây ảnh hưởng nghiêm trọng đến hiệu suất, tính bảo mật và uy tín của sản phẩm. Vì vậy, kiểm thử phần mềm (software testing) đóng vai trò then chốt trong việc đảm bảo hệ thống hoạt động đúng yêu cầu, ổn định và đáng tin cậy trước khi đưa vào sử dụng.

Mục tiêu nghiên cứu của báo cáo là tìm hiểu và hệ thống hóa các khái niệm, mục tiêu, nguyên tắc, quy trình, mô hình, phương pháp và mức độ kiểm thử phần mềm. Báo cáo cũng hướng tới việc phân tích ưu điểm, hạn chế của từng mô hình kiểm thử cũng như làm rõ vai trò của kiểm thử trong việc nâng cao chất lượng sản phẩm. Đồng thời, áp dụng trực tiếp các kiến thức lý thuyết đã nghiên cứu vào tình huống thực tế thông qua việc thực hành kiểm thử trên trang web giả lập.

Phạm vi nghiên cứu tập trung vào các nội dung lý thuyết liên quan đến kiểm thử phần mềm, bao gồm các tiêu chuẩn quốc tế (ISO, IEEE), các phương pháp kiểm thử phổ biến (kiểm thử tĩnh, kiểm thử động, thủ công, tự động), các mô hình phát triển phần mềm kết hợp kiểm thử (Waterfall, V-Model, Iterative – Incremental) và các mức độ kiểm thử (Unit, Integration, System, Acceptance). Và quá trình xây dựng kế hoạch kiểm thử (Test Plan), thiết kế và thực thi các trường hợp kiểm thử (Test Case), ghi nhận và phân tích kết quả, cũng như đánh giá mức độ đáp ứng yêu cầu của hệ thống thông qua việc thực hành trên trang web giả lập phục vụ cho mục đích kiểm thử.

Bố cục bài bài báo cáo gồm các phần chính:

- **Chương 1:** Tổng quan – Trình bày về các lý thuyết về kiểm thử phần mềm và các khái niệm liên quan đến kiểm thử phần mềm.
  1.1. Khái niệm
  1.2. Mục tiêu nghiên cứu
  1.3. Phạm vi nghiên cứu
  1.4. Phương pháp nghiên cứu

- **Chương 2:** Cơ sở lý thuyết – Giới thiệu về trang web thực hành kiểm thử và quy trình thực hiện trên trang web.
  2.1. Giới thiệu website
  2.2. Giới thiệu quy trình hoạt động
- **Chương 3:** Kiểm thử phần mềm – Thiết kế các testcase cho việc kiểm thử phần mềm.
  3.1. Danh sách Testcase
  3.2. Danh sách Bug Report
- **Chương 4:** Kết luận và hướng phát triển – Trình bày về kết quả mà báo cáo đã thực hiện được, các hạn chế và hướng phát triển.
  4.1. Kết quả đạt được
  4.2. Hạn chế và đề xuất cải thiện
  4.3. Hướng phát triển tiếp theo

CHƯƠNG 1: TỔNG QUAN

1.1. Khái niệm

1.1.1. Các khái niệm liên quan

Phần mềm

– Phần mềm (Software) hay còn gọi là phần mềm máy tính (Computer Software), là tập hợp các lệnh hoặc chỉ thị được lập trình bằng một hoặc nhiều ngôn ngữ lập trình. Những lệnh này được tổ chức theo một trình tự xác định, kết hợp với dữ liệu và tài liệu liên quan, nhằm thực hiện tự động các tác vụ, chức năng hoặc giải quyết những vấn đề cụ thể. Tất cả các chương trình hoạt động trên máy tính đều được gọi chung là phần mềm.

– 4 thành phần cơ bản của phần mềm:

- Các chương trình máy tính
- Các thủ tục
- Tài liệu
- Dữ liệu cần thiết để vận hành phần mềm

Chất lượng phần mềm

– Theo IEEE, chất lượng phần mềm được định nghĩa là:

- Mức độ mà một hệ thống, thành phần hoặc quy trình phần mềm đáp ứng được các yêu cầu đã được xác định hoặc đặc tả.
- Mức độ mà hệ thống, thành phần hoặc quy trình phần mềm thỏa mãn nhu cầu và mong đợi của khách hàng hoặc người sử dụng.

– Theo tiêu chuẩn ISO 9126 chất lượng phần mềm được phân chia thành 6 nhóm đặc tính chính:

- Functionality (Tính năng) – Khả năng cung cấp các chức năng đáp ứng nhu cầu đã xác định:
    - Suitability (Tính phù hợp)
    - Accuracy (Độ chính xác)
    - Interoperability (Khả năng tương tác)
    - Security (Bảo mật)

- Compliance (Tuân thủ)

- Reliability (Độ tin cậy) – Khả năng duy trì hiệu suất ổn định trong một khoảng thời gian nhất định:
  - Maturity (Tính ổn định)
  - Fault tolerance (Khả năng chịu lỗi)
- Recoverability (Khả năng khôi phục)
- Usability (Tính dễ sử dụng) – Mức độ dễ dàng để người dùng học và vận hành hệ thống:
  - Understandability (Dễ hiểu)
  - Learnability (Dễ học)
  - Operability (Dễ vận hành)
- Efficiency (Hiệu suất) – Khả năng sử dụng tối ưu tài nguyên hệ thống. Bao gồm:
  - Time behavior (Hiệu suất thời gian)
  - Resource utilization (Hiệu quả sử dụng tài nguyên)
- Maintainability (Tính bảo trì) – Mức độ dễ dàng khi cần sửa đổi hoặc nâng cấp phần mềm:
  - Analyzability (Khả năng phân tích)
  - Changeability (Khả năng thay đổi)
  - Stability (Tính ổn định sau thay đổi)
  - Testability (Khả năng kiểm thử)
- Portability (Tính khả chuyển) – Khả năng chuyển phần mềm sang môi trường hoặc nền tảng khác:
  - Adaptability (Khả năng thích ứng)
  - Installability (Dễ cài đặt)
  - Conformance (Tuân thủ tiêu chuẩn)
  - Replaceability (Khả năng thay thế)

Lỗi phần mềm

– Lỗi phần mềm (hay còn gọi là bug) là những khiếm khuyết hoặc sai lệch trong mã nguồn của một chương trình máy tính, khiến phần mềm không thực hiện đúng chức năng đã được xác định trong đặc tả yêu cầu. Những lỗi này thường phát sinh trong quá trình thiết kế, mã hóa hoặc tích hợp các thành phần phần mềm. Lỗi phần mềm có thể xuất hiện ở bất kỳ giai đoạn nào trong vòng đời phát triển phần mềm và là nguyên nhân chủ yếu dẫn đến các hành vi không mong đợi như: hiển thị sai dữ liệu, hoạt động không ổn định, phản hồi chậm, hoặc thậm chí gây sập hệ thống.

– Các nguyên nhân gây ra lỗi phần mềm:
  - Xác định sai yêu cầu do hiểu sai hoặc ghi nhận không đầy đủ nhu cầu của khách hàng.
  - Thiếu thông tin chính xác.
  - Hiểu nhầm yêu cầu giữa các bên khác nhau về cùng một chức năng.
  - Bất cẩn đến từ phía người lập trình hoặc áp lực thời gian
  - Thiếu kinh nghiệm do không nắm vững kỹ thuật hoặc quy trình phát triển.
  - Ảnh hưởng môi trường do sự khác biệt giữa môi trường phát triển và môi trường chạy thực tế.
  - Lỗi đến từ phía người dùng do sử dụng sai cách, thao tác không đúng với thiết kế phần mềm.

– Tác hại của lỗi phần mềm:
  - Gây tốn thời gian và chi phí cho việc sửa lỗi và bảo trì hệ thống.
  - Làm giảm uy tín và hình ảnh của công ty trong mắt khách hàng.
  - Có thể gây thương tích hoặc tử vong trong các hệ thống quan trọng như y tế, hàng không.
  - Làm gián đoạn hoạt động kinh doanh, ảnh hưởng đến doanh thu.
  - Gây mất dữ liệu quan trọng, ảnh hưởng đến người dùng và tổ chức.
  - Tạo ra lỗ hổng bảo mật, dễ bị khai thác tấn công.
  - Làm mất niềm tin từ người dùng, khiến khách hàng rời bỏ sản phẩm.

Bản yêu cầu của khách hàng

– Requirements Specification (Đặc tả yêu cầu) là tài liệu mô tả chi tiết các yêu cầu về chức năng, phi chức năng và các ràng buộc mà phần mềm cần phải đáp ứng. Đây là

tài liệu này đóng vai trò quan trọng trong việc thiết kế các test case, xác định tiêu chí kiểm thử và đánh giá mức độ hoàn thiện của sản phẩm.

- **Test Plan:**

– Test Plan là một tài liệu được tạo ra dựa trên yêu cầu của dự án, nhằm định hướng và tổ chức hoạt động kiểm thử. Tài liệu này nêu rõ mục tiêu kiểm thử, phạm vi thực hiện, phương pháp tiếp cận, vai trò và trách nhiệm của từng thành viên trong nhóm kiểm thử, đồng thời xác định các tài nguyên cần thiết và kế hoạch xử lý rủi ro phát sinh trong quá trình kiểm thử.

– Việc xây dựng Test Plan và kiểm soát quá trình kiểm thử yêu cầu người thực hiện phải có kiến thức chuyên môn vững và kinh nghiệm thực tế. Do đó, công việc này thường do người quản lý nhóm kiểm thử đảm nhận nhằm đảm bảo việc kiểm thử diễn ra đúng quy trình, đạt chất lượng và đúng tiến độ.

- **Test Case:**

– Test case là một kịch bản kiểm thử cụ thể, mô tả các bước thực hiện, điều kiện tiên quyết, dữ liệu đầu vào và kết quả kỳ vọng. Mục tiêu của test case là kiểm tra xem một chức năng hay thành phần của hệ thống có hoạt động đúng theo yêu cầu đã được xác định hay không. Test case được xây dựng dựa trên các đặc tả yêu cầu về sản phẩm và kế hoạch kiểm thử.

– Các test case được thiết kế phải bảo phủ tất cả các luồng chức năng mô tả trong tài liệu phân tích thiết kế, bao gồm các yêu cầu về bảo mật an toàn thông tin, yêu cầu hiệu năng của hệ thống và các yêu cầu phi chức năng khác.

- **Kiểm thử tĩnh (Static Testing)**

– Khái niệm: Là phương pháp kiểm thử phần mềm được thực hiện mà không cần chạy chương trình hoặc ứng dụng.

– Cách thực hiện: Thông qua việc xem xét, phân tích tài liệu đặc tả, mã nguồn, hoặc thiết kế để phát hiện lỗi sớm.

– Mục tiêu: Xác định các lỗi logic, sai sót trong yêu cầu, hoặc lỗi cú pháp trong mã nguồn ngay từ giai đoạn đầu, giúp giảm chi phí sửa lỗi về sau.

– Đặc điểm:

- Không yêu cầu môi trường thực thi.
- Có thể tiến hành song song với các hoạt động phát triển.

- Giúp cải thiện chất lượng tài liệu và mã nguồn trước khi bước vào kiểm thử động.

Kiểm thử động (Dynamic Testing)

– Khái niệm: Là phương pháp kiểm thử phần mềm thông qua việc thực thi chương trình hoặc ứng dụng trong môi trường thật hoặc giả lập.
– Cách thực hiện: Chạy phần mềm với các dữ liệu kiểm thử đã được thiết kế để quan sát hành vi và kết quả.
– Mục tiêu: Đảm bảo phần mềm hoạt động đúng chức năng, đáp ứng yêu cầu đã xác định và có khả năng xử lý đúng trong các tình huống khác nhau.
– Đặc điểm:
  - Cần môi trường thực thi và dữ liệu kiểm thử.
  - Phát hiện các lỗi về logic xử lý, giao diện, hiệu năng và khả năng tương tác của hệ thống.

Kiểm thử thủ công (Manual Testing)

– Khái niệm: Là phương pháp kiểm thử trong đó người kiểm thử trực tiếp thực hiện các bước kiểm thử mà không sử dụng công cụ tự động.
– Cách thực hiện: Bao gồm quá trình từ thiết kế test case, chuẩn bị dữ liệu đến thực thi kiểm thử và ghi nhận kết quả.
– Mục tiêu: Đánh giá tính đúng đắn và ổn định của phần mềm thông qua việc quan sát và so sánh kết quả thực tế với kết quả mong đợi.
– Đặc điểm:
  - Dễ áp dụng cho các dự án nhỏ hoặc chức năng mới phát triển.
  - Tốn nhiều thời gian và công sức khi số lượng test case lớn hoặc cần kiểm thử lặp lại nhiều lần.
  - Phụ thuộc vào kỹ năng và sự tập trung của tester.

Kiểm thử tự động (Automation Testing)

– Khái niệm: Là phương pháp kiểm thử trong đó các công cụ hoặc phần mềm hỗ trợ sẽ tự động thực hiện các trường hợp kiểm thử đã được định nghĩa trước.
– Cách thực hiện: Người kiểm thử xây dựng các script hoặc mã lệnh để công cụ chạy và so sánh kết quả thực tế với kết quả mong đợi.
– Mục tiêu: Tăng tốc độ kiểm thử, đảm bảo tính nhất quán và giảm sai sót do yếu tố con người, đặc biệt hiệu quả cho các bài kiểm thử lặp lại hoặc kiểm thử hồi quy.

– Đặc điểm:

- Yêu cầu chi phí ban đầu để xây dựng kịch bản và môi trường tự động.
- Hiệu quả cao về lâu dài, đặc biệt trong các dự án lớn hoặc phát triển liên tục.
- Phụ thuộc vào khả năng lựa chọn công cụ và kỹ năng lập trình của đội ngũ kiểm thử

1.1.2. Các khái niệm kiểm thử

Khái niệm:

Kiểm thử phần mềm là quá trình đánh giá một hệ thống phần mềm hoặc các thành phần của nó nhằm xác định xem phần mềm có hoạt động đúng theo yêu cầu đã đặt ra hay không và phát hiện lỗi nếu có. Mục tiêu chính là đảm bảo chất lượng và độ tin cậy của phần mềm trước khi đưa vào sử dụng.

Mục tiêu:

– Phát hiện lỗi và giảm rủi ro

- Tìm ra các lỗi (defect) và các hỏng hóc (failure) của sản phẩm.
- Giúp giảm mức độ rủi ro, nâng cao chất lượng phần mềm.

– Xác minh và đảm bảo rằng tất cả các yêu cầu đã được đáp ứng đầy đủ.

– Kiểm tra tính hoàn chỉnh và xác thực đảm bảo rằng sản phẩm hoạt động đúng như mong đợi của người dùng.

– Xây dựng uy tín nhằm nâng cao sự tin cậy và mức độ chất lượng của sản phẩm.

– Cung cấp thông tin đầy đủ và chính xác cho các bên liên quan để hỗ trợ quá trình ra quyết định, đặc biệt về chất lượng sản phẩm.

– Tuân thủ tiêu chuẩn và yêu cầu

- Đảm bảo sản phẩm phù hợp với yêu cầu, pháp luật hoặc quy định liên quan.
- Xác minh sự tuân thủ với các tiêu chuẩn đã đề ra.

Các mô hình kiểm thử phần mềm

– Mô hình thác nước (Waterfall Model)

- Đặc điểm:
    - Quá trình phát triển phần mềm được thực hiện qua chuỗi các giai đoạn tuần tự: Yêu cầu → Thiết kế → Phát triển → Kiểm thử → Triển khai → Bảo trì.

    - Mỗi giai đoạn hoàn tất trước khi chuyển sang giai đoạn tiếp theo.
  - Ưu điểm:
    - Quy trình rõ ràng, dễ quản lý.
    - Phù hợp với dự án có yêu cầu ổn định, ít thay đổi.
  - Nhược điểm:
    - Khó thích ứng nếu có thay đổi yêu cầu ở giai đoạn sau.
    - Lỗi phát hiện muộn thường tốn kém để sửa.
- Mô hình chữ V (V-Model)
  - Đặc điểm:
    - Là biến thể của mô hình thác nước, nhưng mỗi giai đoạn phát triển đều có giai đoạn kiểm thử tương ứng.
    - Hai nhánh của chữ V: Nhánh trái là các hoạt động phát triển (xác định yêu cầu, thiết kế hệ thống, thiết kế chi tiết). Nhánh phải là các hoạt động kiểm thử (kiểm thử chấp nhận, kiểm thử hệ thống, kiểm thử tích hợp, kiểm thử đơn vị).
  - Ưu điểm:
    - Phát hiện lỗi sớm, giảm chi phí sửa lỗi.
    - Tăng cơ hội thành công nhờ kiểm thử song song với phát triển.
  - Nhược điểm: Ít linh hoạt, khó thay đổi yêu cầu khi dự án đang tiến hành.
- Mô hình tăng trưởng – lặp (Iterative – Incremental Development Model)
  - Đặc điểm:
    - Chia quá trình phát triển thành nhiều vòng lặp (iteration), mỗi vòng bao gồm: Lập kế hoạch → Phân tích & thiết kế → Triển khai → Kiểm thử → Đánh giá.
    - Mỗi vòng lặp tạo ra một phần sản phẩm hoàn chỉnh, dần hoàn thiện hệ thống.
  - Ưu điểm:
    - Phù hợp với dự án phức tạp, yêu cầu thường xuyên thay đổi.
    - Cho phép nhận phản hồi sớm từ người dùng và điều chỉnh kịp thời.

- Hỗ trợ phát triển theo phương pháp Agile, Scrum.

- Nhược điểm:
  - Cần quản lý chặt chẽ để tránh trễ tiến độ do nhiều vòng lặp.
  - Yêu cầu đội ngũ có kỹ năng phối hợp cao.

Các nguyên tắc kiểm thử:

- Kiểm thử chỉ ra sự hiện diện của lỗi: Kiểm thử giúp phát hiện lỗi nhưng không thể chứng minh phần mềm không có lỗi.
- Không thể kiểm thử toàn bộ: Việc kiểm tra tất cả các tình huống là không khả thi do giới hạn về thời gian và tài nguyên.
- Thực hiện kiểm thử càng sớm càng tốt: Phát hiện lỗi sớm giúp giảm chi phí và công sức sửa chữa.
- Lỗi thường tập trung ở một số khu vực: Phần lớn lỗi thường nằm ở một số thành phần cụ thể trong phần mềm.
- Nghịch lý thuốc trừ sâu: Lặp lại cùng một bộ kiểm thử sẽ làm giảm khả năng phát hiện lỗi mới.
- Kiểm thử phụ thuộc vào ngữ cảnh: Mỗi loại phần mềm cần phương pháp kiểm thử phù hợp riêng.
- Không có lỗi không đồng nghĩa với đúng yêu cầu: Phần mềm không có lỗi vẫn có thể không đáp ứng đúng mục tiêu hay nhu cầu người dùng.

Vai trò

- Kiểm thử giúp phát hiện các lỗi trong phần mềm trước khi đưa vào sử dụng, đảm bảo hệ thống hoạt động đúng như mong đợi.
- Đảm bảo phần mềm đáp ứng yêu cầu về chức năng, hiệu năng và tính ổn định, nâng cao độ tin cậy của sản phẩm.
- Giúp kiểm tra phần mềm có được xây dựng đúng theo đặc tả kỹ thuật và phù hợp với nhu cầu của người dùng.
- Việc phát hiện lỗi sớm trong giai đoạn phát triển giúp tiết kiệm chi phí và công sức khắc phục so với khi phần mềm đã triển khai.
- Kết quả kiểm thử là căn cứ để đánh giá phần mềm đã sẵn sàng đưa vào sử dụng hay cần cải thiện thêm.

– Phần mềm ít lỗi và hoạt động mượt mà sẽ tạo ấn tượng tốt và tăng mức độ hài lòng của người sử dụng.
– Thông tin từ quá trình kiểm thử giúp đội ngũ phát triển rút kinh nghiệm và cải thiện quy trình làm việc.

Các mức độ kiểm thử phần mềm

❖ **Kiểm thử đơn vị (Unit Testing)**

– Khái niệm: Kiểm thử đơn vị là mức kiểm thử thấp nhất, tập trung vào việc kiểm tra các thành phần nhỏ nhất có thể kiểm thử trong chương trình như hàm, phương thức, lớp...
– Mục tiêu: Đảm bảo mỗi đơn vị mã nguồn hoạt động đúng theo thiết kế và không chứa lỗi logic nội tại.
– Đặc điểm:
  - Kiểm thử có tính cô lập, không phụ thuộc vào các thành phần khác.
  - Thường được thực hiện song song với quá trình lập trình.
  - Có thể được tự động hóa để thực hiện thường xuyên (test automation).
– Người thực hiện: Lập trình viên.
– Công cụ hỗ trợ phổ biến: JUnit (Java), NUnit (.NET), PyTest (Python), xUnit (C#).

❖ **Kiểm thử tích hợp (Integration Testing)**

– Khái niệm: Sau khi các đơn vị riêng lẻ được kiểm thử thành công, chúng được kết hợp lại và kiểm tra xem việc giao tiếp giữa các module có hoạt động đúng hay không.
– Mục tiêu: Phát hiện lỗi tại giao tiếp giữa các đơn vị: lỗi truyền dữ liệu, gọi hàm sai, hoặc logic phối hợp không đúng.
– Chiến lược kiểm thử tích hợp:
  - Top-down: Kiểm thử từ các module cấp cao xuống cấp thấp, có thể dùng "stub" thay thế cho module cấp dưới chưa sẵn sàng.
  - Bottom-up: Kiểm thử từ các module cấp thấp lên cao, dùng "driver" để mô phỏng gọi từ module cấp trên.
  - Big Bang: Kết hợp tất cả các module rồi kiểm thử toàn bộ một lần (dễ xảy ra lỗi không xác định được vị trí).
– Người thực hiện: Lập trình viên hoặc kiểm thử viên.

❖ **Kiểm thử hệ thống (System Testing)**

- Khái niệm: Đây là mức kiểm thử toàn diện toàn bộ hệ thống phần mềm đã được tích hợp đầy đủ.
- Mục tiêu: Xác minh rằng hệ thống đáp ứng đầy đủ yêu cầu chức năng và phi chức năng đã được mô tả trong tài liệu đặc tả (SRS).
- Phạm vi kiểm thử:
  - Kiểm thử chức năng (Functional Testing): Kiểm tra tính năng hoạt động đúng (đăng nhập, đăng ký, tìm kiếm, xử lý dữ liệu...).
  - Kiểm thử phi chức năng (Non-functional Testing): Kiểm tra hiệu suất, bảo mật, khả năng sử dụng, khả năng mở rộng...
- Người thực hiện: Nhóm QA (kiểm thử viên chuyên nghiệp), độc lập với nhóm phát triển.

❖ **Kiểm thử chấp nhận (Acceptance Testing)**

- Khái niệm: Là bước kiểm thử cuối cùng trước khi triển khai chính thức, nhằm đảm bảo phần mềm phù hợp với mong đợi và nhu cầu thực tế của người dùng.
- Mục tiêu: Đảm bảo hệ thống đã sẵn sàng để được triển khai trong môi trường sản xuất và nhận được sự phê duyệt từ phía khách hàng.
- Phân loại:
  - Alpha Testing: Thực hiện tại nội bộ công ty phát triển bởi đội kiểm thử nội bộ, mô phỏng môi trường thật.
  - Beta Testing: Thực hiện bởi người dùng cuối, tại môi trường sử dụng thực tế, thường trước khi phát hành chính thức, nhằm thu thập phản hồi.
- Người thực hiện: Người dùng đại diện, khách hàng hoặc QA đại diện cho khách hàng.

Một số loại kiểm thử phần mềm:

❖ **Kiểm thử chức năng**

- **Khái niệm:** Là loại kiểm thử nhằm đảm bảo các hành vi của hệ thống thỏa mãn các yêu cầu đặc tả chức năng. Nó được dùng để xác minh rằng phần mềm đang tạo đầu ra như mong đợi của người dùng cuối. Kiểm thử chức năng có xu hướng trả lời câu hỏi của "người sử dụng có thể làm được điều này".
- **Đặc điểm:**
  - Cách tiếp cận hộp đen: Kiểm thử chức năng thường không kiểm tra cấu trúc mã nội bộ mà tập trung vào hành vi của phần mềm từ góc độ bên ngoài.

- Xác minh dựa trên yêu cầu: Các bài kiểm tra được thiết kế để xác minh rằng hệ thống đáp ứng các yêu cầu chức năng đã chỉ định.
- Xác minh đầu vào – đầu ra: Kiểm thử chức năng cung cấp các đầu vào cụ thể và xác minh rằng các đầu ra khớp với các kết quả mong đợi.
- Tập trung vào người dùng: Nó đảm bảo các tính năng hoạt động chính xác từ góc độ người dùng, bất kể cách chúng được thực hiện.

– **Các loại kiểm thử chức năng:**
  - Kiểm thử đơn vị
  - Kiểm thử tích hợp
  - Kiểm tra khói
  - Kiểm thử hồi quy
  - Kiểm thử khám phá

– **Ưu điểm:**
  - Kiểm thử chức năng giúp phát hiện các lỗi hoặc thiếu sót liên quan đến logic nghiệp vụ, xử lý đầu vào và yêu cầu hệ thống ngay từ những giai đoạn đầu, giúp tiết kiệm chi phí sửa lỗi về sau.
  - Kiểm thử chức năng xác thực rằng phần mềm thực hiện đúng các chức năng được mô tả trong tài liệu yêu cầu hoặc đặc tả kỹ thuật, đảm bảo đáp ứng kỳ vọng của người dùng.
  - Bằng cách kiểm tra tính đầy đủ và chính xác của các chức năng, kỹ thuật này giúp phần mềm hoạt động mượt mà, ổn định, từ đó cải thiện trải nghiệm người dùng và tăng độ tin cậy của sản phẩm.
  - Giúp xác minh rằng phần mềm phù hợp với các quy định pháp lý, tiêu chuẩn ngành hoặc yêu cầu về bảo mật, tính toàn vẹn dữ liệu.
  - Kiểm thử chức năng giúp đảm bảo phần mềm hoạt động đúng trên nhiều hệ điều hành, trình duyệt, thiết bị và môi trường khác nhau – tăng tính linh hoạt và phổ quát của ứng dụng.

– **Nhược điểm:**
  - Kiểm thử chức năng chỉ tập trung vào việc xác minh các chức năng theo yêu cầu, mà bỏ qua các khía cạnh quan trọng khác như hiệu năng, bảo mật, khả năng mở rộng và độ ổn định lâu dài của hệ thống.

- Phương pháp này chỉ phát hiện được các vấn đề về chức năng mà không đủ khả năng phát hiện các vấn đề liên quan đến hiệu suất như thời gian phản hồi chậm, sử dụng tài nguyên không hiệu quả hoặc hành vi bất thường của hệ thống khi chịu tải lớn.
- Phụ thuộc nhiều vào chất lượng của test case nên nếu các kịch bản kiểm thử không được xây dựng đầy đủ hoặc thiếu sót, các lỗi nghiêm trọng có thể bị bỏ qua.
- Kiểm thử chức năng không kiểm tra những vấn đề phát sinh khi phần mềm được sử dụng trong thời gian dài, như rò rỉ bộ nhớ, suy giảm hiệu suất, hoặc lỗi tích lũy qua các phiên sử dụng.

Kiểm thử phi chức năng:

– **Khái niệm:**

- Là loại kiểm thử tập trung vào các thuộc tính hoặc các đặc điểm phi chức năng của hệ thống phần mềm, thường là các yếu tố liên quan đến hiệu suất, bảo mật, độ tin cậy, khả năng mở rộng, sự sẵn sàng và các yêu cầu hệ thống khác.
- Mục tiêu của kiểm thử phi chức năng là đảm bảo rằng phần mềm hoạt động không chỉ chính xác về chức năng mà còn đáp ứng các yêu cầu về hiệu suất và các tiêu chí chất lượng khác.

– **Các thông số:**

- Độ tin cậy (Reliability): Khả năng hệ thống hoạt động ổn định, liên tục trong thời gian dài mà không phát sinh lỗi.
- Mức độ bảo mật (Security): Khả năng hệ thống tự bảo vệ khỏi truy cập trái phép, lỗ hổng và các cuộc tấn công mạng.
- Khả năng sử dụng (Usability): Mức độ dễ sử dụng, dễ học và thao tác của người dùng với hệ thống.
- Khả năng mở rộng (Scalability): Khả năng hệ thống đáp ứng hiệu quả khi quy mô xử lý hoặc số lượng người dùng tăng.
- Tính linh hoạt (Flexibility): Khả năng phần mềm hoạt động ổn định trên nhiều cấu hình phần cứng và hệ điều hành khác nhau.
- Khả năng tương tác (Interoperability): Khả năng hệ thống kết nối và làm việc với các hệ thống phần mềm khác.

- Khả năng tái sử dụng (Reusability): Khả năng sử dụng lại các mô-đun hoặc thành phần phần mềm trong các hệ thống khác.
- Tính di động (Portability): Khả năng linh hoạt của phần mềm có thể chuyển đổi từ môi trường phần cứng hoặc phần mềm hiện tại sang môi trường khác.
- Tính khả dụng (Availability): Biểu thị mức độ mà người dùng có thể truy cập và sử dụng hệ thống một cách liên tục.
- Tính hiệu suất (Efficiency): Mức độ mà hệ thống sử dụng tài nguyên phần cứng và phần mềm một cách hiệu quả để hoàn thành nhiệm vụ.

– **Các loại kiểm thử phi chức năng:**

- Kiểm thử hiệu năng (Performance testing)
- Kiểm thử khả năng chịu tải (Load testing)
- Kiểm thử áp lực (Stress testing)
- Kiểm thử tính khả dụng (Usability testing)
- Kiểm thử bảo trì (Maintainability testing)
- Kiểm thử độ tin cậy (Reliability testing)
- Kiểm thử tính tương thích (Portability testing)

Kiểm thử hộp đen

– **Khái niệm:**

- Kiểm thử hộp đen (Black Box Testing) là phương pháp kiểm thử phần mềm tập trung vào kiểm tra các chức năng của phần mềm mà không cần biết cấu trúc bên trong của mã nguồn. Trong quá trình kiểm thử này, người kiểm thử tập trung vào đầu vào và đầu ra dự kiến mà không quan tâm đến cách phần mềm xử lý dữ liệu hoặc logic nội bộ.
- Mục tiêu của kiểm thử hộp đen là xác minh phần mềm có hoạt động đúng theo yêu cầu chức năng hay không, thông qua việc kiểm tra đầu vào và đầu ra mà không cần biết mã nguồn bên trong. Phương pháp này giúp phát hiện lỗi xử lý dữ liệu, điều kiện biên, và đảm bảo phần mềm đáp ứng đúng hành vi mong đợi từ góc nhìn người dùng hoặc hệ thống bên ngoài.

– **Một số kỹ thuật kiểm thử hộp đen:**

- Kiểm thử hồng ngoại (Boundary Testing)

- Kiểm thử Tuyến tính (Equivalence Partitioning)
- Kiểm thử Về sau (Regression Testing)
- Kiểm thử Hậu nghiệm (Smoke Testing)
- Kiểm thử Mô phỏng (Scenario Testing)

– **Ưu điểm:**
  - Kiểm thử hộp đen tập trung vào chức năng và trải nghiệm thực tế theo góc nhìn của người dùng cuối, đảm bảo phần mềm hoạt động đúng theo yêu cầu.
  - Tester có thể thực hiện kiểm thử mà không cần hiểu biết về mã nguồn, phù hợp với cả những người không chuyên về lập trình.
  - Do nhóm kiểm thử độc lập thực hiện, giúp tăng tính khách quan và giảm thiên lệch trong quá trình đánh giá chất lượng.
  - Dễ phát hiện các lỗi người dùng thường gặp mà lập trình viên có thể bỏ sót do quen thuộc với logic nội bộ.
  - Rất hữu ích trong kiểm thử hệ thống, kiểm thử tích hợp, kiểm thử hồi quy và kiểm thử chấp nhận người dùng (UAT).
  - Có thể xây dựng kịch bản kiểm thử sớm dựa trên tài liệu yêu cầu, không phụ thuộc vào tiến độ phát triển mã.
  - Giúp xác minh phần mềm đã thực hiện đầy đủ và chính xác các chức năng theo đặc tả.

– **Nhược điểm:**
  - Phải tạo nhiều trường hợp đầu vào để đảm bảo độ bao phủ chức năng, gây tốn thời gian và công sức.
  - Tài liệu yêu cầu thiếu thông tin rõ ràng khiến việc thiết kế test case gặp khó khăn, dễ xảy ra thiếu sót.
  - Do không biết cấu trúc nội bộ, tester khó đánh giá đã kiểm thử đủ các đường dẫn quan trọng hay chưa.
  - Chỉ một phần nhỏ đầu vào được kiểm thử, nhiều nhánh xử lý bên trong có thể không được đánh giá.
  - Dễ phát sinh các test case trùng lặp hoặc kiểm tra dư thừa, gây lãng phí nguồn lực.

- Thiếu định hướng rõ ràng, thiếu tầm nhìn về hoạt động nội tại của phần mềm nên phụ thuộc nhiều vào phỏng đoán và thử nghiệm.

Kiểm thử hộp trắng

– **Khái niệm:**

- Là một phương pháp kiểm tra phần mềm tập trung để xác định các thành phần nội bộ của phần mềm như: dữ liệu cấu trúc, chế độ điều khiển logic và mã hóa các thành phần nguồn. Người kiểm thử cần có kiến thức lập trình để viết và thực hiện các ca kiểm thử, nhằm đảm bảo mọi đường đi logic trong chương trình đều được kiểm tra.
- Mục tiêu là kiểm tra tính đúng đắn của các đường đi bên trong mã nguồn chứ không chỉ dựa vào yêu cầu đầu vào/đầu ra như trong kiểm thử hộp đen.

– **Các phương pháp kiểm thử hộp trắng:**

- Kiểm tra mã dòng (phạm vi bao phủ mã)
- Kiểm tra nhánh (phạm vi bao phủ nhánh)
- Kiểm tra điều kiện (phạm vi bao phủ điều kiện)
- Kiểm tra dữ liệu (phạm vi bao phủ luồng dữ liệu)

– **Ưu điểm:**

- Kiểm thử hộp trắng cho phép kiểm tra sâu vào cấu trúc bên trong mã nguồn, bao phủ đầy đủ các nhánh điều kiện, vòng lặp và đường đi trong chương trình, giúp phát hiện các lỗi logic mà kiểm thử hộp đen có thể bỏ sót, từ đó đảm bảo tính toàn diện cho hệ thống.
- Vì dựa trên mã nguồn, kiểm thử hộp trắng có thể thực hiện ngay từ những giai đoạn đầu, giúp phát hiện lỗi sớm, giảm chi phí khắc phục lỗi và hạn chế rủi ro phát sinh trong các giai đoạn sau.
- Hỗ trợ phát hiện đoạn mã dư thừa, không thực thi (dead code) hoặc thiếu hiệu quả, giúp tối ưu hóa mã nguồn, góp phần cải thiện hiệu suất, tính rõ ràng và khả năng bảo trì của hệ thống.
- Với độ bao phủ cao và chi tiết, kiểm thử hộp trắng nâng cao mức độ ổn định và chính xác, đặc biệt trong các hệ thống yêu cầu nghiêm ngặt về chất lượng.
- Có thể tự động hóa và tích hợp vào pipeline CI/CD, giúp kiểm soát chất lượng liên tục và giảm thiểu lỗi tái phát trong quá trình phát triển phần mềm.

– **Nhược điểm:**

- Kiểm thử hộp trắng yêu cầu người kiểm thử phải có hiểu biết sâu về kiến trúc hệ thống và mã nguồn, nên không phù hợp với các nhóm kiểm thử thiếu nền tảng kỹ thuật hoặc không chuyên về lập trình.
- Việc phân tích chi tiết và thiết kế test case cho từng nhánh mã trong kiểm thử hộp trắng đòi hỏi thời gian dài và công sức lớn, đặc biệt trong các hệ thống có mã nguồn phức tạp hoặc quy mô lớn.
- Do tập trung vào mức mã nguồn, kiểm thử hộp trắng không hiệu quả trong việc phát hiện lỗi về thiết kế giao diện, yêu cầu nghiệp vụ, hoặc cấu trúc kiến trúc tổng thể.
- Đối với hệ thống lớn với nhiều mô-đun tương tác, kiểm thử chi tiết từng phần mã trở nên khó khăn và dễ bỏ sót lỗi tích hợp giữa các thành phần.
- Khi mã nguồn thay đổi, các test case hộp trắng dễ bị lỗi thời và cần cập nhật thường xuyên, dẫn đến chi phí bảo trì cao trong các dự án có tốc độ phát triển nhanh.

1.1.3. Kỹ thuật TestCase

Phân tích giá trị biên

– Phân tích giá trị biên là phương pháp kiểm thử tập trung vào các giá trị nằm ở biên của các khoảng dữ liệu đầu vào và đầu ra. Thay vì phải kiểm thử tất cả các giá trị trong phạm vi, người kiểm thử chỉ cần kiểm thử các giá trị ở biên để đảm bảo hệ thống hoạt động chính xác. Phương pháp này giúp giảm số lượng test case nhưng vẫn đảm bảo độ bao phủ.

– Boundary conditions là các vị trí ở giữa, trên và dưới các biên của lớp tương đương. Khi áp dụng kỹ thuật phân tích giá trị biên, người kiểm thử sẽ chọn các giá trị:

- Giá trị nhỏ nhất
- Giá trị ngay dưới giá trị nhỏ nhất
- Giá trị bình thường
- Giá trị lớn nhất
- Giá trị ngay trên giá trị lớn nhất

– Ưu điểm:

- Phân tích giá trị biên giúp tăng tính đúng đắn nhờ việc kiểm tra phần mềm trong các trường hợp đặc biệt và gần giới hạn dữ liệu, tăng độ tin cậy của phần mềm.
- Phân tích giá trị biên giúp tiết kiệm thời gian vì chỉ cần kiểm thử các giá trị gần biên thay vì toàn bộ tập giá trị đầu vào, giảm thiểu số lượng test case cần thực hiện.
- Đây là một kỹ thuật kiểm thử đơn giản, dễ áp dụng trong thực tế.
- Giúp phát hiện các lỗi xảy ra tại ranh giới dữ liệu – nơi thường phát sinh nhiều lỗi nhất.

– Nhược điểm:
  - Không kiểm tra toàn bộ dữ liệu đầu vào do kỹ thuật chỉ tập trung vào các giá trị quan trọng gần biên, bỏ sót các giá trị ở giữa.
  - Không đảm bảo kiểm thử toàn diện vì nó có thể bỏ qua lỗi xuất hiện ở các giá trị không nằm gần biên, cần kết hợp với các kỹ thuật khác để tăng độ phủ kiểm thử.
  - Yêu cầu hiểu biết về kiểu dữ liệu vì kỹ thuật này cần có kiến thức chuyên môn để xác định ranh giới phù hợp với kiểu dữ liệu và ngữ cảnh kiểm thử.

Phân vùng tương đương

– Phân vùng tương thích là kỹ thuật chia đầu vào thành các nhóm có tính chất tương đương. Mỗi nhóm sẽ đại diện cho một loại kết quả, do đó nếu một giá trị trong nhóm hoạt động đúng, thì tất cả các giá trị khác trong nhóm cũng sẽ hoạt động đúng, và ngược lại. Việc phân chia này giúp giảm thiểu số lượng test case cần thiết.

– Thiết kế Test-case bằng phân vùng tương đương tiến hành theo 2 bước:
  - Bước 1: Xác định các lớp tương đương.
  - Bước 2: Xác định các ca kiểm thử.

– Khi thực hiện kỹ thuật Equivalence partitioning, đầu vào sẽ được chia theo nguyên tắc:
  - 1 lớp các giá trị lớn hơn.
  - 1 lớp các giá trị nhỏ hơn.
  - 1 lớp các giá trị hợp lệ.

– Ưu điểm:

- Kỹ thuật phân vùng tương đương giúp giảm số lượng test casecần thực hiện mà vẫn đảm bảo kiểm tra các tình huống quan trọng, từ đó tiết kiệm thời gian và chi phí kiểm thử.
- Kỹ thuật này giúp tăng hiệu quả kiểm thử vì nó chỉ tập trung vào các ca kiểm thử đại diện giúp nâng cao hiệu quả phát hiện lỗi hơn so với việc kiểm thử toàn bộ dữ liệu đầu vào.
- Đây là một kỹ thuật dễ áp dụng trong thực tế, phù hợp với nhiều loại phần mềm.
- Giúp tiết kiệm chi phí và thời gian vì số lượng test case ít hơn nên thời gian thực hiện ngắn hơn, chi phí kiểm thử cũng được giảm thiểu.

– Nhược điểm:

- Kỹ thuật phân vùng tương đương không đảm bảo kiểm thử toàn diện vì nó không thể bao phủ toàn bộ các trường hợp có thể xảy ra, dễ bỏ sót lỗi nếu phân vùng không chính xác.
- Kỹ thuật này phụ thuộc vào khả năng phân tích, đòi hỏi người kiểm thử có kỹ năng phân tích tốt để xác định đúng các vùng tương đương.
- Đối với các chức năng có nhiều giá trị đầu vào và mối quan hệ phức tạp, việc xác định các vùng tương đương có thể gặp khó khăn và dễ sai sót.

Ad – hoc

– Kỹ thuật ad hoc (Ad-hoc testing) là một phương pháp kiểm thử phần mềm không chính thức, không có kế hoạch hoặc tài liệu trước, mà người kiểm thử sử dụng sự sáng tạo và kinh nghiệm của mình để tìm ra lỗi.

– Đặc điểm của kiểm thử ad hoc:

- Không có kế hoạch trước như: không có các bước kiểm thử được xác định trước, tài liệu hướng dẫn, hoặc trường hợp kiểm thử cụ thể.
- Người kiểm thử sử dụng kiến thức và kinh nghiệm của mình để tìm ra các lỗi tiềm ẩn mà các phương pháp kiểm thử chính thức có thể bỏ sót.
- Có thể được thực hiện bất cứ lúc nào và không yêu cầu nhiều thời gian chuẩn bị.
- Giúp phát hiện ra các lỗi mà các phương pháp kiểm thử khác có thể bỏ qua, đặc biệt là trong các dự án có ngân sách nhỏ hoặc thời gian hạn chế.

– Ưu điểm:

- Giúp tìm ra các lỗi và vấn đề tiềm ẩn một cách nhanh chóng và hiệu quả.

- Thích hợp cho các dự án có quy mô nhỏ, ngân sách hạn chế hoặc thời gian gấp rút.
- Cho phép người kiểm thử sử dụng sự sáng tạo và kinh nghiệm của mình để tìm ra các lỗi tiềm ẩn.
- Nâng cao, cải thiện test cases, giúp tạo ra các trường hợp thử nghiệm tốt hơn và giúp chất lượng sản phẩm tốt hơn.

– Nhược điểm:
  - Do không có kế hoạch hoặc tài liệu cụ thể, việc tái hiện lỗi có thể gặp khó khăn.
  - Không có tài liệu cụ thể có thể gây khó khăn trong việc theo dõi và giải quyết các lỗi.
  - Phụ thuộc rất nhiều vào kinh nghiệm và khả năng của người kiểm thử.
  - Khó phát hiện các vấn đề tiềm tàng trong quá trình kiểm thử.

Biểu đồ chuyển trạng thái

– Kỹ thuật chuyển đổi trạng thái là một kỹ thuật kiểm thử Hộp đen (Black-box testing), được áp dụng khi hệ thống có thể được mô hình hóa dưới dạng một tập hữu hạn các trạng thái, và các hành vi của hệ thống được điều khiển bởi các quy tắc chuyển đổi giữa các trạng thái.

– Hệ thống phản ứng dựa trên đầu vào (input) và trạng thái hiện tại, từ đó dẫn đến trạng thái mới. Việc kiểm thử tập trung vào việc xác minh rằng hệ thống hoạt động chính xác khi chuyển từ trạng thái này sang trạng thái khác, thông qua các đầu vào cụ thể.

– Các thành phần của một biểu đồ trạng thái:
  - Các trạng thái phần mềm là một tình huống hoặc điều kiện của một hệ thống. Nó được biểu diễn bằng một nút trong sơ đồ chuyển đổi trạng thái. Mỗi nút biểu diễn một trạng thái khác nhau của hệ thống.
  - Chuyển đổi từ trạng thái này sang trạng thái khác là sự thay đổi trạng thái của hệ thống xảy ra khi hệ thống phản ứng với một sự kiện được biểu diễn bằng mũi tên.
  - Các sự kiện là một hành động hoặc sự việc gây ra sự thay đổi trạng thái của hệ thống, được dán nhãn phía trên mũi tên chuyển tiếp áp dụng.
  - Các hành động phát sinh từ quá trình chuyển đổi từ một sự kiện là hành vi mà hệ thống thể hiện khi nó thay đổi trạng thái, có thể được biểu diễn bằng hộp tin nhắn.

– Ưu điểm:

- Cung cấp cái nhìn trực quan về hệ thống: Kỹ thuật này mô tả hành vi của hệ thống dưới dạng hình ảnh (sơ đồ trạng thái) hoặc bảng trạng thái, giúp tester dễ dàng hình dung và hiểu cách hệ thống vận hành.
- Bao phủ trạng thái và điều kiện tốt: Giúp xác minh rằng tất cả các trạng thái hợp lệ và không hợp lệ cũng như các điều kiện chuyển đổi đều được kiểm tra đầy đủ.
- Hỗ trợ phát hiện lỗi trong luồng xử lý: Có khả năng xác định và ngăn ngừa lỗi xảy ra trong quá trình chuyển đổi giữa các trạng thái của hệ thống.
- Cải thiện chất lượng phần mềm: Việc kiểm thử chi tiết các luồng chuyển đổi giúp tăng độ tin cậy và chất lượng tổng thể của hệ thống.

– Nhược điểm:

- Chỉ áp dụng cho hệ thống hữu hạn trạng thái: Phương pháp này chỉ phù hợp với những hệ thống có thể mô hình hóa bằng một số lượng trạng thái hữu hạn rõ ràng.
- Không phù hợp với hệ thống không tuần tự: Những hệ thống không có luồng xử lý theo thứ tự tuần tự sẽ khó áp dụng phương pháp này một cách hiệu quả.
- Khó triển khai với hệ thống lớn và phức tạp: Việc xác định và mô hình hóa tất cả các trạng thái có thể trở nên rất phức tạp và tốn thời gian khi hệ thống có nhiều trạng thái hoặc điều kiện rẽ nhánh.
- Không đảm bảo phát hiện mọi lỗi tiềm ẩn: Dù bao phủ tốt các trạng thái và điều kiện, nhưng kỹ thuật này có thể vẫn bỏ sót một số lỗi tiềm ẩn không liên quan đến chuyển trạng thái.

Bảng quyết định

– Kiểm tra bảng quyết định là kỹ thuật hữu ích trong trường hợp các điều kiện đầu vào có sự kết hợp phức tạp. Bằng cách sử dụng bảng quyết định, người kiểm thử có thể xác định được tất cả các tình huống kết hợp của đầu vào và xác định hành động tương ứng cho mỗi tình huống. Điều này giúp giảm số lượng test case trong khi vẫn đảm bảo độ bao phủ tối đa.

– Quy trình tạo bảng quyết định gồm 4 bước:

- Liệt kê tất cả Conditions/Inputs.
- Tính số lượng kết hợp có thể (Rules).

- Đặt tất cả các kết hợp trong bảng.
- Giảm thiểu các case kết hợp và quyết định Test Case.

– Ưu điểm:
  - Bảng quyết định giúp mô hình hóa các quy tắc kinh doanh phức tạp một cách rõ ràng, trực quan, hỗ trợ việc tổng quan hóa và review toàn bộ chức năng.
  - Các điều kiện và hành động được biểu diễn rõ ràng trong bảng, giúp các bên liên quan như QA, PM, khách hàng nắm bắt được logic một cách trực quan, minh bạch.
  - Hỗ trợ xây dựng các test case có tính hệ thống, đảm bảo bao phủ các tổ hợp điều kiện cần thiết mà không bị bỏ sót.
  - Giúp phát hiện các trường hợp kiểm thử bị thiếu hoặc không được thực hiện trong quá trình phát triển và kiểm thử.
  - Việc xây dựng bảng quyết định góp phần làm sáng tỏ yêu cầu, đồng thời phát hiện những yêu cầu không hợp lý hoặc mâu thuẫn.
  - Khi đã xây dựng bảng quyết định đầy đủ, việc tạo test case trở nên nhanh chóng và hiệu quả hơn.

– Nhược điểm:
  - Việc xây dựng bảng quyết định chi tiết đòi hỏi nhiều thời gian và công sức, đặc biệt với các hệ thống có nhiều điều kiện và quy tắc.
  - Bảng quyết định chỉ đưa ra kết quả logic (true/false) giúp đánh giá các yếu tố về chức năng, không đánh giá được các yếu tố phi chức năng như bảo mật, hiệu suất, khả năng chịu tải,…
  - Người thiết kế và sử dụng bảng cần có kiến thức và kỹ năng phân tích tốt, do việc xử lý các bảng có nhiều điều kiện đầu vào có thể gây khó khăn cho người mới.
  - Khi số lượng điều kiện tăng lên, số tổ hợp có thể trở nên rất lớn, khiến việc kiểm tra tất cả các trường hợp là không khả thi nếu bị giới hạn về thời gian hoặc nguồn lực.
  - Bất kỳ thay đổi nào trong yêu cầu phần mềm đều phải được cập nhật vào bảng quyết định, làm tăng chi phí bảo trì và nỗ lực quản lý tài liệu kiểm thử.
  - Mặc dù hữu ích, bảng quyết định không thể thay thế hoàn toàn cho các test case cụ thể trong việc kiểm thử chi tiết.

Đoán lỗi

- Đoán lỗi là một kỹ thuật kiểm thử phần mềm dựa trên việc đoán lỗi có thể chiếm ưu thế trong code. Đây là một kỹ thuật dựa trên kinh nghiệm, trong đó nhà phân tích kiểm thử sử dụng kinh nghiệm của mình để đoán phần có vấn đề hoặc có lỗi của ứng dụng kiểm thử. Thay vì tuân theo các quy tắc cụ thể, kỹ thuật này tập trung vào việc khai thác kinh nghiệm và kiến thức của người kiểm thử để tạo ra các test case tìm kiếm lỗi.
- Nguyên tắc đoán lỗi:
  - Ghi nhớ các lỗi rắc rối trước đây để làm tài liệu tham khảo cho các lần kiểm thử sau. Điều này giúp nhận diện nhanh các lỗi thường gặp trong các loại ứng dụng tương tự.
  - Tester cần tìm hiểu cách viết mã và cách vận hành của các khái niệm như con trỏ null, mảng, chỉ mục, giới hạn, vòng lặp,… để hiểu rõ hơn về nguồn gốc và nguyên nhân gây ra lỗi trong phần mềm.
  - Hiểu rõ môi trường kỹ thuật triển khai ứng dụng như: máy chủ, hệ điều hành, cơ sở dữ liệu nơi ứng dụng được triển khai để đánh giá được các vấn đề có thể phát sinh do môi trường, không chỉ do code.
  - Người tester cần có khả năng phát hiện các điểm chưa hợp lý hoặc thiếu sót trong yêu cầu nghiệp vụ, tài liệu thiết kế và các trường hợp kiểm thử. Việc phát hiện vấn đề từ sớm sẽ giúp tiết kiệm thời gian và chi phí chỉnh sửa sau này.
  - Hiểu rõ được kiến trúc và luồng hoạt động chính của hệ thống để xây dựng các kịch bản kiểm thử sát với thực tế sử dụng.
  - Đánh giá dữ liệu lịch sử và kết quả kiểm thử để rút ra các quy luật phát sinh lỗi và đánh giá được mức độ ổn định của hệ thống qua các phiên bản.
  - Tester cần duy trì nhận thức và cập nhật thường xuyên các loại lỗi phổ biến trong lĩnh vực kiểm thử để tăng hiệu suất kiểm tra và phát hiện lỗi tiềm ẩn.
- Ưu điểm:
  - Hiệu quả khi kết hợp với các kỹ thuật kiểm thử chính thức. Error Guessing phát huy hiệu quả tốt nhất khi được áp dụng song song với các kỹ thuật kiểm thử khác, giúp tăng độ phủ kiểm thử.

- Những lỗi tiềm ẩn không được mô tả rõ hoặc bị bỏ sót trong tài liệu vẫn có thể được tìm ra nhờ kinh nghiệm của tester.
- Tiết kiệm thời gian và công sức cho tester giàu kinh nghiệmCác giả định hợp lý và tập trung giúp tester nhanh chóng định hướng khu vực dễ phát sinh lỗi.
- Giúp khám phá những vùng rủi ro cao của phần mềm vì tập trung kiểm thử vào các khu vực có khả năng xảy ra lỗi cao, nhờ vào kinh nghiệm và phán đoán của tester.
- Hỗ trợ thiết kế thêm nhiều testcase hữu ích, các testcase không được phát hiện qua các kỹ thuật kiểm thử có hệ thống.

– Nhược điểm:
  - Kỹ thuật đoán lỗi phụ thuộc nhiều vào kinh nghiệm cá nhân nên chỉ hiệu quả nếu người thực hiện là tester có kiến thức sâu và kinh nghiệm thực tế. Người mới rất khó áp dụng.
  - Đây là một kỹ thuật cảm tính, không có quy trình cụ thể hoặc tiêu chuẩn nhất định nên dễ dẫn đến thiếu sót nếu không cẩn thận.
  - Không đảm bảo phạm vi kiểm thử tối thiểu nên dễ bỏ sót các tình huống kiểm thử quan trọng do không có cấu trúc rõ ràng.
  - Việc đoán lỗi lan man, không có cơ sở hoặc thiếu tập trung có thể dẫn đến thiết kế test case không hiệu quả và tốn thời gian mà không tìm ra lỗi.

1.1.4. Lỗi phần mềm

1.1.4.1. Lỗi yêu cầu

a) Tiếp cận – Mô tả

– Lỗi yêu cầu là lỗi xảy ra trong quá trình thu thập, phân tích hoặc đặc tả yêu cầu, khiến phần mềm được phát triển không đúng hoặc không đầy đủ theo nhu cầu thực tế của người dùng. Những lỗi này thường không xuất phát từ quá trình lập trình hay kiểm thử, mà từ sự hiểu sai hoặc thiếu sót ngay từ giai đoạn đầu của vòng đời phần mềm.

– Ví dụ các lỗi yêu cầu phổ biến:
  - Đặc tả thiếu chức năng quan trọng.
  - Yêu cầu không rõ ràng, mâu thuẫn giữa các yêu cầu.
  - Không cập nhật yêu cầu khi có thay đổi thực tế.

- Giao diện đáp ứng sai hành vi mong đợi vì hiểu sai logic người dùng.
- Yêu cầu không phù hợp với nghiệp vụ thực tế của doanh nghiệp.

b) Nguyên nhân

– Nhận thức không đầy đủ của người dùng hoặc khách hàng về nhu cầu thực tế.
– Phán đoán chủ quan của người phân tích nghiệp vụ (BA), hiểu sai mục tiêu dự án.
– Thiếu trao đổi giữa các bên liên quan (stakeholders, người dùng, nhà phát triển).
– Thiếu kỹ năng phân tích hoặc đặc tả yêu cầu theo chuẩn.
– Thiếu thời gian hoặc vội vàng trong giai đoạn phân tích.
– Không có tài liệu đặc tả hoặc đặc tả không được xác nhận chính thức.

c) Ghi nhận (Minh chứng)

– Người dùng phản hồi rằng phần mềm không đáp ứng đúng nghiệp vụ.
– Tester phát hiện chức năng thực tế khác với mô tả trong tài liệu đặc tả.
– So sánh giữa tài liệu yêu cầu và hệ thống đã xây dựng.
– Thay đổi yêu cầu sau khi hệ thống đã hoàn tất phần lớn tính năng (rework).
– Ghi nhận sự sai khác khi thực hiện walkthrough hoặc review tài liệu yêu cầu.

d) Khắc phục

– Business Analyst/System Analyst: Thu thập yêu cầu đầy đủ, rõ ràng và chính xác từ người dùng; áp dụng các phương pháp mô hình hóa nghiệp vụ (Use Case, BPMN, Activity Diagram...) để phân tích, xác minh và truyền đạt yêu cầu một cách nhất quán giữa các bên liên quan.
– Developer (Lập trình viên): Phối hợp chặt chẽ với BA để hiểu rõ mục tiêu và phạm vi của yêu cầu; chủ động phản hồi khi phát hiện điểm chưa rõ ràng hoặc mâu thuẫn trong tài liệu; tuân thủ yêu cầu đã xác nhận và báo cáo sai lệch nếu phát sinh trong quá trình phát triển.
– Tester kỹ thuật: Tham gia sớm vào giai đoạn phân tích yêu cầu; thực hiện review tài liệu để phát hiện điểm mơ hồ, thiếu sót; thiết kế test case bám sát yêu cầu nhằm xác minh tính đúng đắn và đầy đủ của chức năng hệ thống.
– Khách hàng/Người dùng đại diện (Stakeholder): Tham gia xác nhận yêu cầu từ giai đoạn đầu và phản hồi kịp thời khi có điều chỉnh hoặc phát hiện sai sót; hỗ trợ kiểm chứng lại nội dung sau khi yêu cầu được cập nhật hoặc chỉnh sửa.

1.1.4.2. Lỗi thiết kế

a) Tiếp cận – Mô tả

– Lỗi thiết kế là những sai sót trong việc phân tích, mô hình hóa hoặc xác định yêu cầu hệ thống trong giai đoạn đầu của quá trình phát triển phần mềm. Đây là lỗi nền tảng, có thể ảnh hưởng tới toàn bộ kiến trúc, chức năng và hiệu năng của hệ thống, dẫn đến sai sót lan truyền trong các giai đoạn tiếp theo như lập trình, kiểm thử, vận hành. Những lỗi này thường đến từ:

- Mô hình hệ thống không đầy đủ hoặc không phản ánh đúng nghiệp vụ thực tế
- Thiết kế chức năng mâu thuẫn, thiếu tính khả thi hoặc không phù hợp yêu cầu người dùng
- Bỏ sót các trường hợp biên, điều kiện ngoại lệ trong luồng xử lý
- Thiết kế giao diện hoặc tương tác không thân thiện, gây nhầm lẫn cho người dùng
- Phân quyền và bảo mật chưa được tích hợp từ đầu trong thiết kế

b) Nguyên nhân

– Hiểu sai hoặc không đầy đủ yêu cầu người dùng trong quá trình thu thập yêu cầu.
– Thiếu kinh nghiệm trong phân tích hệ thống hoặc thiết kế kiến trúc phần mềm.
– Thiết kế dựa trên giả định thay vì xác thực từ tài liệu hoặc người dùng thực tế.
– Không sử dụng mô hình hóa (UML, ERD...) để xác minh tính logic và đầy đủ của thiết kế.
– Không tổ chức review thiết kế, dẫn đến lỗi không được phát hiện sớm.
– Không tính đến khả năng mở rộng, bảo trì hoặc tích hợp trong tương lai.

c) Ghi nhận (Minh chứng):

– Tính năng hoạt động nhưng không đúng với mục đích sử dụng ban đầu của người dùng
– Nhiều thay đổi phải thực hiện trong giai đoạn lập trình vì phát hiện lỗi từ thiết kế
– Chức năng phát triển xong nhưng bị bỏ do không còn phù hợp với nghiệp vụ
– Kiểm thử thất bại ở mức tích hợp hoặc hệ thống do lỗi logic từ thiết kế gốc
– Người dùng phản ánh thao tác phức tạp, khó sử dụng, luồng xử lý không hợp lý
– Báo cáo thiết kế bị thiếu mô tả, sơ đồ hoặc thông tin quan trọng dẫn đến hiểu sai trong triển khai

d) Khắc phục

- Business Analyst/System Analyst: Thu thập yêu cầu đầy đủ, chính xác từ người dùng; sử dụng mô hình hóa nghiệp vụ (BPMN, Use Case, Activity Diagram...) để phân tích và xác minh yêu cầu.
- Kiến trúc sư phần mềm (Software Architect): Thiết kế kiến trúc hệ thống hợp lý, linh hoạt, đảm bảo hiệu suất và khả năng mở rộng; tính đến các yếu tố bảo mật, phân quyền, tương thích nền tảng.
- Developer (Lập trình viên): Phối hợp chặt chẽ với BA để hiểu đúng yêu cầu thiết kế, phản hồi kịp thời các điểm không rõ ràng; tuân thủ thiết kế chuẩn và báo cáo sai lệch trong quá trình phát triển.
- Tester kỹ thuật: Tham gia sớm vào quá trình review tài liệu thiết kế; thiết kế test case dựa trên yêu cầu để phát hiện sớm mâu thuẫn hoặc sai sót trong thiết kế chức năng và luồng xử lý.

1.1.4.3. Lỗi mã hóa/lập trình

a) Tiếp cận – Mô tả:

- Lỗi mã hóa/lập trình là những sai sót xảy ra trong quá trình hiện thực hóa (coding) phần mềm – nơi các chức năng được chuyển thành mã nguồn. Những lỗi này xuất phát từ việc lập trình sai logic, thiếu điều kiện xử lý, sai cú pháp hoặc sử dụng sai cấu trúc ngôn ngữ. Đây là lỗi kỹ thuật trực tiếp ảnh hưởng đến hoạt động, hiệu năng và tính ổn định của phần mềm.
- Các lỗi thường gặp:
  - Xử lý điều kiện sai logic (nếu/else, vòng lặp,...)
  - Truy cập tài nguyên không kiểm tra null hoặc quyền.
  - Gán giá trị sai kiểu, vượt giới hạn mảng.
  - Không xử lý ngoại lệ (exception).
  - Không đồng bộ luồng trong ứng dụng đa luồng.
  - Dùng sai hàm, sai thứ tự gọi API, hoặc vi phạm chuẩn mã hóa.

b) Nguyên nhân:

- Lập trình viên không hiểu đầy đủ yêu cầu hoặc thiết kế ban đầu.
- Thiếu kiểm tra logic, xử lý không đầy đủ các trường hợp biên hoặc ngoại lệ.

- Cấu trúc mã rối, thiếu tổ chức, không tuân thủ nguyên tắc lập trình sạch.
- Không viết unit test để kiểm tra chức năng nhỏ.
- Làm việc dưới áp lực hoặc thiếu quy trình review mã nguồn.

c) Ghi nhận (Minh chứng):

- Chức năng hoạt động không đúng, trả về kết quả sai hoặc lỗi không mong muốn.
- Ứng dụng crash do lỗi runtime như NullPointerException, TypeError, IndexOutOfBoundsException.
- Kết quả xử lý không nhất quán giữa các phiên bản hoặc môi trường.
- Unit test hoặc integration test thất bại vì sai logic.
- Người dùng phản hồi thao tác không đúng như mô tả chức năng.
- Log hệ thống hiển thị lỗi biên dịch, lỗi xử lý hoặc lỗi dữ liệu phát sinh từ mã nguồn.

d) Khắc phục

- Developer (Lập trình viên): Viết mã theo chuẩn coding convention, xử lý đầy đủ các điều kiện logic, ngoại lệ và trường hợp biên. Thực hiện kiểm thử đơn vị (unit test), tích hợp (integration test) và tự review mã trước khi commit. Phối hợp chặt với BA và tester để xác minh yêu cầu.
- Code Reviewer / Senior Developer: Thực hiện review mã nguồn định kỳ để phát hiện lỗi logic hoặc sai sót cú pháp. Hướng dẫn junior developer về kỹ thuật và best practices. Kiểm tra các nguy cơ gây lỗi bảo mật, hiệu năng thấp.
- Tester kỹ thuật: Thiết kế test case bao phủ logic phức tạp, test giá trị biên, test dữ liệu đầu vào không hợp lệ. Ghi nhận rõ hành vi sai lệch, hỗ trợ xác định nguyên nhân lỗi đến từ mã nguồn.
- DevOps / Automation QA: Tích hợp kiểm thử tự động vào pipeline CI/CD, thiết lập kiểm thử hồi quy, cảnh báo sớm lỗi sau mỗi lần cập nhật mã. Dùng các công cụ như SonarQube để phân tích chất lượng mã nguồn và cảnh báo sớm về mã nguy hiểm hoặc lỗi logic tiềm tàng.

1.1.4.4. Lỗi dữ liệu

a) Tiếp cận – Mô tả

- Lỗi dữ liệu là những sai sót xảy ra trong quá trình nhập, xử lý, lưu trữ, truyền tải hoặc hiển thị dữ liệu trong hệ thống phần mềm. Đây là lỗi liên quan đến tính hợp lệ, đầy

đủ, chính xác và nhất quán của dữ liệu, thường phát sinh do kỹ thuật hoặc xử lý nội bộ của hệ thống, không phải do người dùng cuối trực tiếp gây ra. Những lỗi này có thể nằm ở:

- Cấu trúc và ràng buộc của cơ sở dữ liệu
- Logic xử lý dữ liệu trong mã nguồn
- Giao tiếp và truyền tải dữ liệu giữa các thành phần hệ thống
- Cơ chế chuyển đổi định dạng hoặc mã hóa dữ liệu
- Thiếu cơ chế kiểm tra tính đúng đắn, hợp lệ của dữ liệu trong hệ thống

b) Nguyên nhân

- Thiết kế cơ sở dữ liệu không chặt chẽ: Thiếu ràng buộc như NOT NULL, UNIQUE, CHECK, khóa chính hoặc khóa ngoại.
- Lập trình xử lý dữ liệu không đúng: Không kiểm tra hoặc làm sạch dữ liệu trước khi lưu vào hệ thống.
- Không chuẩn hóa dữ liệu: Dữ liệu được tổ chức không khoa học, dư thừa, gây trùng lặp hoặc mâu thuẫn.
- Lỗi trong quá trình truyền nhận dữ liệu giữa các hệ thống: Dữ liệu bị cắt, sai kiểu, hoặc sai định dạng.
- Lỗi mã hóa (encoding): Không đồng bộ bảng mã giữa frontend, backend hoặc CSDL, gây lỗi ký tự hiển thị.
- Thiếu kiểm thử dữ liệu (data testing): Không có quy trình kiểm tra dữ liệu đầu ra hoặc sau xử lý.

c) Ghi nhận (Minh chứng)

- Dữ liệu hiển thị sai hoặc không đầy đủ trên giao diện người dùng so với dữ liệu gốc trong cơ sở dữ liệu.
- Kết quả xử lý không chính xác do dữ liệu thiếu, sai định dạng, trùng lặp hoặc không nhất quán giữa các bảng.
- Lỗi truy vấn dữ liệu: Hệ thống trả về lỗi SQL do dữ liệu null, thiếu khóa chính/ngoại hoặc xung đột ràng buộc dữ liệu.
- Kiểm thử chức năng thất bại: Các test case đầu vào hợp lệ nhưng trả kết quả sai vì dữ liệu test bị lỗi.

- Phản hồi từ người dùng: Người dùng phát hiện thông tin cá nhân, báo cáo, số liệu thống kê không chính xác.
- Log hệ thống hoặc báo cáo lỗi: Hiển thị thông tin dữ liệu bị lỗi, bị bỏ qua, hoặc không thể xử lý.

d) Khắc phục

- Lập trình viên (Developer): Xử lý và xác thực dữ liệu trong mã nguồn, kiểm tra logic nhập/xuất, tránh lưu trữ dữ liệu sai định dạng hoặc trùng lặp.
- Kỹ sư cơ sở dữ liệu (Database Designer/DBA): Thiết kế cơ sở dữ liệu chuẩn hóa, áp dụng ràng buộc dữ liệu (NOT NULL, UNIQUE, FOREIGN KEY) và kiểm tra dữ liệu định kỳ.
- Kỹ sư tích hợp hệ thống: Đồng bộ định dạng và cấu trúc dữ liệu giữa các hệ thống tích hợp, xử lý sai lệch dữ liệu khi chuyển đổi và truyền tải. Đảm bảo dữ liệu đồng bộ khi trao đổi giữa các hệ thống.
- Tester kỹ thuật: Thiết kế test case đa dạng để phát hiện lỗi dữ liệu, thực hiện kiểm thử dữ liệu đầu vào/đầu ra và xác minh tính nhất quán dữ liệu trong toàn hệ thống.

1.1.4.5. Lỗi môi trường

a) Tiếp cận – Mô tả

- Lỗi môi trường là những sai sót phát sinh từ môi trường triển khai hoặc vận hành hệ thống phần mềm, bao gồm phần cứng, hệ điều hành, nền tảng trung gian, trình duyệt, mạng, hoặc các công cụ hỗ trợ (middleware). Đây là lỗi không nằm trong mã nguồn ứng dụng, mà đến từ nơi mà phần mềm được cài đặt, triển khai, thực thi hoặc truy cập. Những lỗi này có thể nằm ở:
  - Phiên bản hệ điều hành hoặc trình duyệt không tương thích
  - Cấu hình máy chủ (server) không phù hợp
  - Thiếu thư viện, thành phần phụ thuộc hoặc biến môi trường
  - Lỗi bảo mật, quyền truy cập hoặc firewall
  - Sự cố mạng: DNS, tốc độ, timeout, cắt kết nối
  - Khác biệt giữa môi trường dev – test – staging – production

b) Nguyên nhân

- Thiết lập cấu hình sai trong môi trường triển khai (ví dụ: cấu hình proxy, port bị xung đột).
- Phụ thuộc vào thư viện hoặc gói phần mềm nhưng hệ thống thiếu hoặc sai phiên bản (ví dụ: thiếu NodeJS, Java, .NET runtime).
- Khác biệt giữa các hệ điều hành (Linux, Windows) dẫn đến lỗi đường dẫn, phân biệt chữ hoa/thường, ký tự đặc biệt...
- Web server (Apache, Nginx, IIS...) cấu hình sai hoặc thiếu module cần thiết.
- Cơ sở hạ tầng không đáp ứng: thiếu bộ nhớ, CPU, không đủ quyền truy cập vào tệp hệ thống.
- Thay đổi trong môi trường (cập nhật hệ điều hành, vá bảo mật, cập nhật trình duyệt) ảnh hưởng đến khả năng chạy hệ thống.
- Tường lửa, antivirus hoặc hệ thống bảo mật ngăn chặn kết nối hoặc hành vi hợp lệ của ứng dụng.
- Hệ thống phụ trợ (database, API, Redis, Kafka...) không hoạt động, không kết nối hoặc trả dữ liệu sai.

c) Ghi nhận (Minh chứng)

- Cài đặt hoặc triển khai thất bại: Log hệ thống hiển thị lỗi thiếu thư viện, sai quyền truy cập hoặc thành phần phụ thuộc chưa được cài đặt.
- Ứng dụng hoạt động không ổn định: Chạy tốt trên môi trường phát triển nhưng xảy ra lỗi (crash, sai kết quả) khi triển khai thực tế.
- Kiểm thử chéo môi trường: Ứng dụng hoạt động đúng trên trình duyệt Chrome nhưng lỗi trên Firefox (cross-browser) hoặc sai khác giữa Windows và Linux (cross-platform).
- Theo dõi log hệ thống: Sử dụng công cụ như systemd, Docker logs, hoặc cloud logs để phát hiện lỗi DNS, phân quyền, kết nối hoặc tài nguyên.
- So sánh cấu hình môi trường: Phát hiện sai khác giữa biến môi trường hoặc file cấu hình ở các môi trường phát triển, kiểm thử và triển khai.
- Giám sát tài nguyên hệ thống: Tình trạng quá tải CPU, RAM hoặc timeout thường xuyên chỉ ra lỗi liên quan đến tài nguyên không phù hợp trong môi trường.

d) Khắc phục

- DevOps / Kỹ sư triển khai: Chuẩn hóa quy trình thiết lập môi trường bằng công cụ tự động hóa (CI/CD, Docker), đảm bảo nhất quán cấu hình và tài nguyên giữa các môi trường.
- System Administrator: Quản lý hạ tầng phần cứng, hệ điều hành, mạng và quyền truy cập. Đảm bảo môi trường vận hành ổn định và đáp ứng đúng yêu cầu hệ thống.
- Developer: Đảm bảo mã nguồn không phụ thuộc cứng vào môi trường, có cấu hình linh hoạt.
- Tester kỹ thuật: Thực hiện kiểm thử trên nhiều môi trường khác nhau, ghi nhận điều kiện kiểm thử cụ thể và phát hiện sớm sự không tương thích do môi trường gây ra.

1.1.4.6. Lỗi kiểm thử

a) Tiếp cận – Mô tả

- Lỗi kiểm thử là sai sót xảy ra trong quá trình thiết kế, thực hiện hoặc đánh giá kết quả kiểm thử phần mềm. Đây không phải là lỗi của hệ thống, mà là lỗi do chính người kiểm thử hoặc công cụ kiểm thử gây ra, làm ảnh hưởng đến chất lượng đánh giá phần mềm, có thể dẫn đến bỏ sót lỗi thật hoặc báo sai lỗi.

b) Nguyên nhân:

- Hiểu sai yêu cầu phần mềm khi thiết kế test case.
- Thiết kế test case thiếu các tình huống quan trọng (đặc biệt là biên).
- Thực hiện kiểm thử không đúng trình tự, sai môi trường hoặc sai dữ liệu.
- Đánh giá sai kết quả kiểm thử, không phát hiện lỗi hoặc báo lỗi giả.
- Kịch bản kiểm thử tự động (automation script) có lỗi logic.
- Tài liệu kiểm thử lỗi thời, thiếu cập nhật hoặc không đầy đủ.

c) Ghi nhận (Minh chứng)

- Test Case sai hoặc không đầy đủ: Kiểm thử không bao phủ các trường hợp biên, điều kiện ngoại lệ hoặc sai logic nghiệp vụ dẫn đến bỏ sót lỗi nghiêm trọng.
- Phát hiện lỗi muộn: Lỗi nghiêm trọng được phát hiện ở giai đoạn tích hợp hoặc vận hành, cho thấy kiểm thử chưa hiệu quả ở các mức đơn vị hoặc chức năng.
- Kết quả kiểm thử không ổn định: Test case lúc pass, lúc fail với cùng dữ liệu đầu vào – có thể do test script không kiểm soát đúng trạng thái hệ thống hoặc phụ thuộc vào dữ liệu không được reset.

- Báo cáo kiểm thử không chính xác: Ghi nhận kết quả sai, đánh dấu pass cho các chức năng thực tế đang lỗi; thiếu ảnh chụp màn hình, log hoặc bước tái hiện lỗi.
- Kiểm thử hồi quy (Regression Testing) không đầy đủ: Sau khi cập nhật phần mềm, các chức năng cũ phát sinh lỗi nhưng không được phát hiện do thiếu test case hồi quy.
- Thiếu kiểm thử tích hợp (Integration Testing): Các module hoạt động tốt riêng lẻ nhưng xảy ra lỗi khi kết hợp do thiếu kịch bản kiểm thử liên kết các thành phần.

d) Khắc phục

- Tester: Nâng cao kỹ năng viết test case, kiểm tra chéo với đồng đội, bám sát yêu cầu.
- Automation QA: Viết script rõ ràng, kiểm tra kỹ logic và dữ liệu test, bảo trì script thường xuyên.
- Test Leader: Lên kế hoạch chi tiết, phân công hợp lý, kiểm soát tiến độ và chất lượng kiểm thử.
- Business Analyst (BA): Cung cấp tài liệu yêu cầu rõ ràng, giải thích đầy đủ cho nhóm kiểm thử.
- Công cụ kiểm thử: Cấu hình chính xác, cập nhật phiên bản mới, kiểm tra môi trường chạy định kỳ.

1.1.4.7. Lỗi tài liệu

a) Tiếp cận – Mô tả

- Lỗi tài liệu xảy ra khi các tài liệu liên quan đến dự án (đặc tả yêu cầu, tài liệu thiết kế, hướng dẫn sử dụng, test case, v.v.) bị sai lệch, thiếu thông tin, mâu thuẫn hoặc không cập nhật kịp thời, gây hiểu nhầm hoặc triển khai sai trong quá trình phát triển và kiểm thử phần mềm. Lỗi có thể xảy ra ở mọi loại tài liệu:
  - Tài liệu yêu cầu (Requirement Specification)
  - Tài liệu thiết kế (Design Document)
  - Tài liệu hướng dẫn sử dụng (User Manual, Installation Guide)
  - Tài liệu kiểm thử (Test Case, Test Plan, Test Report)
  - Tài liệu kỹ thuật (Technical Spec, API Document, Config Guide)

b) Nguyên nhân

- Yêu cầu thay đổi nhưng tài liệu không được cập nhật kịp thời.
- Giao tiếp không hiệu quả giữa các bên (BA, Dev, Tester...).

- Thiếu quy trình kiểm tra chéo và phê duyệt tài liệu.
- Thiếu tiêu chuẩn thống nhất trong cách viết và trình bày tài liệu.
- Viết vội vàng hoặc không đầy đủ do áp lực thời gian.

c) Ghi nhận (Minh chứng)

- Lập trình viên hoặc kiểm thử viên nhận thấy tài liệu không phản ánh đúng hành vi hệ thống như đã triển khai hoặc yêu cầu từ người dùng.
- Kịch bản kiểm thử cho kết quả không chính xác do nội dung mô tả sai nghiệp vụ hoặc điều kiện kiểm thử không phù hợp.
- Người dùng gặp khó khăn khi thao tác do hướng dẫn sử dụng không rõ ràng hoặc gây hiểu nhầm.
- Nhân viên triển khai không thể thiết lập hệ thống đúng cách do thiếu tài liệu hướng dẫn hoặc thông tin sai lệch.
- Tài liệu kỹ thuật API (ví dụ: Swagger, Postman collection) không khớp với cấu trúc và tham số thực tế của API.
- Trong quá trình rà soát, tài liệu bị phát hiện thiếu tiêu chuẩn trình bày như định dạng không thống nhất, ký hiệu không đúng quy ước hoặc thiếu thông tin quan trọng.

d) Khắc phục

- Business Analyst (BA): Xác minh đầy đủ yêu cầu với khách hàng và các bên liên quan; đảm bảo tài liệu rõ ràng, nhất quán, có cập nhật khi thay đổi; bổ sung ví dụ, sơ đồ hoặc mockup để diễn giải dễ hiểu.
- Tester: Chủ động phát hiện và phản hồi các điểm mơ hồ trong tài liệu ngay từ giai đoạn đọc hiểu; lập checklist các điểm chưa rõ để trao đổi nhóm; đề xuất bổ sung mô tả nếu không đủ thông tin kiểm thử.
- Developer: Không tự suy đoán khi tài liệu chưa rõ ràng; kịp thời báo cáo mâu thuẫn hoặc thiếu sót; phối hợp chặt chẽ với BA và tester để xác nhận đúng yêu cầu hệ thống.

Báo cáo lỗi

- Báo cáo lỗi (Bug Report) là một tài liệu ghi lại chi tiết về lỗi hoặc sự cố phát sinh trong quá trình kiểm thử phần mềm. Mục tiêu của báo cáo lỗi là cung cấp đầy đủ thông tin để lập trình viên có thể hiểu, tái hiện và sửa lỗi một cách hiệu quả. Việc viết báo cáo lỗi rõ ràng, chính xác và đầy đủ là một kỹ năng quan trọng của người kiểm

thử, góp phần đảm bảo chất lượng phần mềm và hỗ trợ hiệu quả cho nhóm phát triển trong quá trình xử lý lỗi.

- Các thành phần của một báo cáo lỗi:
  - Mã lỗi (Bug ID)
  - Tên lỗi (Title)
  - Mô tả (Description)
  - Các bước thực hiện (Steps)
  - Kết quả mong đợi (Expected)
  - Kết quả thực tế (Actual)
  - Trạng thái (Status)
  - Minh chứng (Screenshot)
- Các loại báo cáo lỗi cơ bản:
  - Báo cáo lỗi chức năng: Mô tả lỗi liên quan đến chức năng phần mềm.
  - Báo cáo lỗi hiệu năng: Liên quan đến tốc độ, khả năng phản hồi hoặc mức tiêu thụ tài nguyên của phần mềm.
  - Báo cáo lỗi bảo mật: Phản ánh các lỗ hổng có thể bị khai thác trái phép, gây ảnh hưởng đến an toàn hệ thống và dữ liệu.
  - Báo cáo lỗi giao diện (UI/UX): Mô tả lỗi gây ảnh hưởng đến giao diện hoặc trải nghiệm người dùng như bố cục sai lệch, chức năng giao diện không hoạt động.

1.2. Mục tiêu nghiên cứu

- Đánh giá chất lượng và khả năng đáp ứng yêu cầu của website Automation Test Store thông qua quá trình kiểm thử phần mềm, bao gồm:
  - Quản lý tài khoản: đăng ký, đăng nhập, chỉnh sửa thông tin, đổi mật khẩu, đăng xuất, xem lịch sử đơn hàng.
  - Quản lý danh mục và sản phẩm: truy cập danh mục, tìm kiếm, sắp xếp, xem chi tiết sản phẩm.
  - Quản lý mua hàng: thêm sản phẩm vào giỏ hàng, chỉnh sửa giỏ hàng, thanh toán, xác nhận và theo dõi đơn hàng.
- Phát hiện, ghi nhận và phân loại các lỗi chức năng (sai logic, điều kiện biên, dữ liệu đầu ra không đúng, điều hướng sai luồng).

– Thực hành và áp dụng các kiến thức, kỹ thuật kiểm thử phần mềm đã học để thiết kế test case, thực thi kiểm thử và báo cáo kết quả.

1.3. Phạm vi nghiên cứu

– Đối tượng kiểm thử: Website dành cho việc kiểm thử Automation Test Store tại địa chỉ https://automationteststore.com.
– Phạm vi chức năng: Các chức năng cơ bản của một website thương mại điện tử, bao gồm:
  - Quản lý tài khoản (đăng ký, đăng nhập, chỉnh sửa thông tin, đổi mật khẩu, đăng xuất, xem lịch sử đơn hàng).
  - Quản lý danh mục và sản phẩm (xem danh mục, tìm kiếm, sắp xếp, xem chi tiết sản phẩm).
  - Quản lý mua hàng (thêm vào giỏ hàng, chỉnh sửa giỏ hàng, thanh toán, xác nhận đơn hàng, theo dõi đơn hàng).
– Phạm vi phi chức năng: Tính khả dụng, khả năng phản hồi, xử lý dữ liệu và hiển thị thông tin.

1.4. Phương pháp nghiên cứu

1.4.1. Phương pháp định tính

– Quan sát trực tiếp giao diện và cách thức vận hành của hệ thống trên môi trường web.
– Ghi nhận chi tiết các yếu tố như bố cục trang, vị trí các thành phần giao diện, cấu trúc menu, nội dung hiển thị và thông tin phản hồi của hệ thống khi người dùng thực hiện các hành động (ví dụ: đăng nhập, đăng ký, tìm kiếm sản phẩm, thêm vào giỏ hàng, thanh toán).
– Đánh giá mức độ dễ sử dụng, tính logic của luồng thao tác và mức độ đáp ứng các yêu cầu nghiệp vụ đặt ra.

1.4.2. Phương pháp định lượng

– Thống kê số lượng ca kiểm thử (test case) đã xây dựng, số bước kiểm thử trong mỗi ca và tỷ lệ ca đạt/không đạt sau khi thực hiện.
– Ghi nhận thời gian phản hồi của hệ thống cho từng thao tác quan trọng như tải trang chi tiết sản phẩm, thực hiện tìm kiếm, xử lý thanh toán.

1.4.3. Phương pháp phân tích

- Phân tích luồng nghiệp vụ chi tiết của các chức năng trọng tâm, bao gồm: quản lý tài khoản, tìm kiếm sản phẩm, duyệt danh mục, thêm sản phẩm vào giỏ hàng, thanh toán và theo dõi đơn hàng.
- Xác định các điểm nghẽn hoặc bất cập trong quá trình xử lý, ví dụ: bước thừa, thông tin nhập lặp lại, hoặc lỗi hiển thị (nếu có).

1.4.4. Phương pháp thử nghiệm (Testing)

- Thiết kế test case dựa trên yêu cầu nghiệp vụ và các kịch bản sử dụng thực tế.
- Thực hiện kiểm thử chức năng (Functional Testing) để xác minh từng tính năng hoạt động đúng như mô tả; kiểm thử giao diện (UI Testing) để đảm bảo các thành phần hiển thị đúng và tương thích; và kiểm thử luồng người dùng (User Flow Testing) để đánh giá trải nghiệm tổng thể.
- Ghi lại kết quả kiểm thử kèm minh chứng bằng ảnh chụp màn hình, thông báo lỗi hoặc trích xuất log từ hệ thống.

1.4.5. Phương pháp đối chiếu – so sánh

- So sánh kết quả thực tế với kết quả mong đợi được xác định trong test case.
- Đối chiếu với các tiêu chuẩn và thông lệ tốt trong thiết kế giao diện người dùng, thương mại điện tử và bảo mật dữ liệu.
- Từ đó, đưa ra các nhận xét, đánh giá và khuyến nghị cụ thể để cải thiện hệ thống.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Giới thiệu website

Automation Test Store là một cửa hàng thương mại điện tử mô phỏng được thiết kế dành cho mục đích học và thực hành kiểm thử phần mềm. Website này tái tạo các thao tác tiêu chuẩn trong quy trình mua sắm trực tuyến như đăng ký tài khoản, đăng nhập, tìm kiếm sản phẩm, xem chi tiết, đưa sản phẩm vào giỏ, thực hiện thanh toán và theo dõi đơn hàng. Ngoài ra, trang web còn cung cấp các biểu mẫu đầu vào, thông báo phản hồi, xử lý điều hướng và nhiều tình huống thực tế để phục vụ mục tiêu kiểm thử phần mềm. Mặc dù có đầy đủ chức năng như một trang mua sắm thực thụ, hệ thống không thực hiện xử lý thanh toán thật hoặc chuyển đơn đến người dùng. Thay vào đó, người dùng nên sử dụng dữ liệu giả để thử nghiệm thông qua trang web này.

Link truy cập: https://automationteststore.com

Trang chủ website gồm các thành phần sau:

- Phần đầu trang (Header)
  - Logo và tên hệ thống được hiển thị ở góc trái, kèm biểu tượng giỏ hàng.
  - Công cụ quản lý và điều hướng nhanh: Bao gồm tùy chọn đăng nhập/đăng ký, truy cập tài khoản, xem giỏ hàng, thực hiện thanh toán, và liên kết đến trang sản phẩm giảm giá (Specials).
  - Tùy chọn tiền tệ: Cho phép thay đổi đơn vị tiền tệ hiển thị.
  - Thanh tìm kiếm: Hỗ trợ người dùng tìm sản phẩm theo từ khóa.
- Thanh điều hướng chính (Navigation bar)
  - Liệt kê các danh mục sản phẩm như: Apparel & Accessories, Makeup, Skincare, Fragrance, Men, Hair Care và Books và kèm với liên kết để có thể quay lại Home
- Khu vực nội dung chính (Main Content)
  - Banner giới thiệu: Nêu rõ mục đích của website là hỗ trợ rèn luyện kỹ năng kiểm thử thông qua môi trường giả lập.
  - Các tiện ích chính: Bao gồm thông tin về vận chuyển nhanh, thanh toán thuận tiện, tùy chọn giao hàng và sự đa dạng sản phẩm.

- Thông báo quan trọng: Khẳng định website chỉ dùng cho mục đích giáo dục, không xử lý đơn hàng hoặc thanh toán thật.
- Khu vực hiển thị sản phẩm gồm: Featured Products, Latest Products, Bestsellers, Specials, Brands Scrolling List

– Phần chân trang (Footer) gồm các mục:
  - About us: Trình bày mục đích và phạm vi hoạt động của website.
  - Contact us: Số điện thoại và email hỗ trợ.
  - Testimonials: Một số nhận xét tiêu biểu từ khách hàng.
  - Newsletter signup: Form đăng ký để nhận thông tin khuyến mãi và tin tức mới.
  - Thanh điều hướng gồm:
    - Bên phải gồm các tùy chọn: About us, Privacy Policy, Return Policy, Shipping, Contact us, Site map, Login.
    - Bên trái là liên kết mạng xã hội: Các liên kết đến Facebook, Twitter, và LinkedIn.

2.2. Giới thiệu quy trình hoạt động

2.2.1. Quản lý tài khoản

Website cung cấp đầy đủ các chức năng quản lý tài khoản như: tạo tài khoản mới, đăng nhập, cập nhật thông tin cá nhân, thay đổi mật khẩu, đăng xuất và tra cứu lịch sử đơn hàng đã mua. Cụ thể:

– Tạo tài khoản mới
  - Người dùng truy cập liên kết “Login or Register” ở góc trên bên phải giao diện trang web.
  - Trong khu vực “New Customer”, chọn “Continue” để mở biểu mẫu đăng ký.
  - Biểu mẫu yêu cầu nhập các thông tin cá nhân và tài khoản bao gồm: First Name, Last Name, Email Address, Telephone, Address, City, Region/State, Zip/Postal Code, Country, Login Name, Password và Password Confirm.
  - Sau khi hoàn tất, nhấn “Continue” để gửi thông tin. Nếu dữ liệu hợp lệ, hệ thống sẽ tạo tài khoản và tự động đăng nhập cho người dùng.

– Đăng nhập hệ thống

- Với tài khoản đã đăng ký, người dùng chọn "Login or Register", nhập Login Name (**thanhhien**) và Password (**123456**) trong mục "Returning Customer", sau đó nhấn "Login".
- Nếu thông tin chính xác, hệ thống điều hướng tới trang "My Account"; nếu sai, sẽ hiển thị thông báo lỗi.

– Cập nhật thông tin cá nhân: Sau khi đăng nhập, người dùng chọn "Edit account details" để thay đổi các thông tin như họ tên, email, số điện thoại hoặc địa chỉ nhận hàng.
– Thay đổi mật khẩu: Tại mục "Change your password", người dùng nhập mật khẩu mới và xác nhận lại trước khi lưu thay đổi.
– Đăng xuất: Khi kết thúc phiên làm việc, người dùng nhấn "Logout" để thoát khỏi hệ thống.
– Tra cứu lịch sử đơn hàng: Trong mục "Order History" tại trang tài khoản, hệ thống hiển thị danh sách các đơn hàng đã thực hiện, bao gồm: mã đơn hàng, ngày đặt, trạng thái xử lý và tổng giá trị.

Ngoài ra trang web còn cung cấp thêm các chức năng như:

– "Transaction History": Chức năng này lưu trữ các thông tin về những giao dịch tài chính đã phát sinh trong tài khoản người dùng gồm: "ID" (Mã định danh của giao dịch), "Date Created" (Ngày tạo giao dịch), "Type" (Loại giao dịch), "Debit" (Số tiền ghi nợ), "Credit" (Số tiền ghi có), "Description" (Mô tả chi tiết giao dịch).
– "Downloads": Lưu lại thông tin tải về các đơn hàng.
– "Notifications": Tùy chọn nhận hay không nhận các thông báo về "Newsletters"(Các tin tức, cập nhật về sản phẩm và dịch vụ), "Order Update" (Thông báo về tình trạng đơn hàng), "Account Update" (Thông báo về các thay đổi hoặc cập nhật liên quan đến tài khoản) thông qua email.

2.2.2. Quản lý danh mục sản phẩm và sản phẩm

2.2.2.1. Truy cập danh mục sản phẩm

Người dùng có thể chọn nhanh các nhóm sản phẩm như "Skincare", "Fragrance", "Makeup", "Hair Care", "Men", "Books", hoặc "Apparel & Accessories" thông qua

thanh menu nằm ở phía trên giao diện. Khi một danh mục được chọn, hệ thống sẽ hiển thị trang chứa toàn bộ sản phẩm thuộc danh mục đó.

Ở phần trên cùng, một số nhóm con (ví dụ: *Conditioner*, *Shampoo* trong danh mục Hair Care) được hiển thị kèm hình ảnh để người dùng dễ truy cập.

Danh sách sản phẩm được trình bày theo dạng lưới hoặc danh sách, mỗi sản phẩm nằm trong một khối riêng gồm:

- Hình ảnh minh họa sản phẩm
- Tên sản phẩm
- Giá bán
- Nút thêm vào giỏ hàng hoặc nhãn "Out of Stock" nếu sản phẩm hết hàng.

Khi số lượng sản phẩm vượt quá giới hạn hiển thị, hệ thống cung cấp pagination ở cuối trang để người dùng chuyển sang các trang tiếp theo. Ngoài ra, người dùng còn có thể tùy chọn số lượng sản phẩm hiển thị trên mỗi trang.

2.2.2.2. Sắp xếp sản phẩm

Trong trang danh mục, người dùng có thể sắp xếp danh sách sản phẩm có sẵn nhờ thanh công cụ cho phép sắp xếp sản phẩm theo các tiêu chí:

- Date old > New
- Name A – Z
- Name Z – A
- Price low > High
- Price high > Low
- Rating Highest
- Rating Lowest

Việc sắp xếp giúp người dùng dễ dàng tìm thấy sản phẩm phù hợp theo nhu cầu cá nhân.

2.2.2.3. Tìm kiếm sản phẩm theo từ khóa

Bên cạnh chức năng duyệt theo danh mục, hệ thống còn hỗ trợ thanh tìm kiếm ở góc trên trang web, cho phép người dùng nhập từ khóa bất kỳ để tra cứu sản phẩm trên toàn

bộ cửa hàng. Khi người dùng nhấn Enter hoặc chọn biểu tượng tìm kiếm, hệ thống sẽ trả về danh sách các sản phẩm có tên hoặc mô tả trùng khớp với từ khóa.

Nếu không tìm thấy kết quả phù hợp, hệ thống sẽ hiển thị thông báo: *"There is no product that matches the search criteria."* nhằm thông báo cho người dùng biết rằng không có sản phẩm nào đáp ứng tiêu chí tìm kiếm.

2.2.2.4. Xem chi tiết sản phẩm

Khi người dùng lựa chọn tên hoặc hình ảnh của một sản phẩm bất kỳ trong danh mục, hệ thống sẽ điều hướng đến trang chi tiết sản phẩm. Tại đây, các thông tin được trình bày đầy đủ và trực quan, bao gồm:

- Tên đầy đủ của sản phẩm
- Hình ảnh về sản phẩm
- Mô tả chi tiết về đặc điểm, thành phần, công dụng và các thông tin liên quan.
- Giá bán của sản phẩm.
- Tình trạng tồn kho, cho biết sản phẩm còn hàng hay đã hết hàng.
- Trường nhập số lượng để người dùng lựa chọn số lượng đặt mua.
- Nút "Add to Cart" cho phép thêm sản phẩm vào giỏ hàng để tiếp tục quá trình mua sắm.

Ngoài ra trang web còn hiển thị thêm các sản phẩm gần đây để người dùng có thể tham khảo.

2.2.3. Quản lý mua sản phẩm

- Thêm sản phẩm vào giỏ hàng
  - Tại trang chi tiết sản phẩm, người dùng nhập số lượng mong muốn và nhấn nút "Add to Cart".
  - Trang web chuyển đến trang giỏ hàng để người dùng có thể kiểm tra và chỉnh sửa giỏ hàng (nếu cần).
- Xem và chỉnh sửa giỏ hàng
  - Người dùng nhấp vào biểu tượng giỏ hàng để xem danh sách sản phẩm đã chọn.
  - Tại đây, người dùng có thể:

    - Thay đổi số lượng sản phẩm.
    - Xóa sản phẩm khỏi giỏ hàng.
    - Xem tổng giá trị đơn hàng.
  - Sau khi chỉnh sửa, nhấn "Update" để áp dụng các thay đổi.

- Tiến hành thanh toán (Checkout)
  - Từ trang giỏ hàng, người dùng chọn "Checkout" để bắt đầu quy trình thanh toán.
  - Nếu chưa đăng nhập, hệ thống sẽ yêu cầu người dùng đăng nhập hoặc đăng ký tài khoản trước khi tiếp tục.
  - Hệ thống sẽ chuyển đến trang "CHECKOUT CONFIRMATION" để người dùng kiểm tra và chỉnh sửa các thông tin như: tên người nhận, số điện thoại, địa chỉ giao hàng (nếu cần). Hệ thống hỗ trợ tính năng tự động điền (prefill) thông tin nhận hàng dựa trên dữ liệu đã được lưu trong hồ sơ người dùng, giúp rút ngắn thời gian thao tác và hạn chế sai sót khi nhập liệu.
- Xác nhận đơn hàng
  - Sau khi kiểm tra, người dùng nhấn "Confirm Order" để hoàn tất.
  - Hệ thống hiển thị thông báo đặt hàng thành công kèm mã đơn hàng (Order ID).
- Theo dõi đơn hàng
  - Người dùng có thể truy cập mục "Order History" để xem chi tiết các đơn hàng đã đặt, bao gồm: mã đơn hàng, ngày đặt, trạng thái và tổng giá trị.

CHƯƠNG 3: KIỂM THỬ PHẦN MỀM

3.1. Danh sách Testcase

3.1.1. Quản lý tài khoản

Bảng 3. 1: Danh sách Testcase chức năng Register

TC ID	Function: Register		
RE_01	**Description:** Đăng ký tài khoản thành công khi người dùng điền đầy đủ các trường thông tin hợp lệ và đồng ý điều khoản. **Pre-condition:** • Người dùng chưa có tài khoản. • Login name và email sử dụng để đăng ký chưa tồn tại trong hệ thống. • Truy cập được trang "Login or Register" của website https://automationteststore.com **Steps:** • Truy cập trang web: https://automationteststore.com/index.php?rt=account/login • Tại vùng "I AM A NEW CUSTOMER", chọn Continue. • Tại form Register Account, nhập đầy đủ các trường thông tin bắt buộc. • Tick chọn checkbox "I have read and agree to the Privacy Policy". • Chọn Continue. **Test Data:**	**Test reuslt:** Passed	**Note:** Kiểm tra khả năng xác thực thông tin hợp lệ trước khi tiến hành đăng ký tài khoản

	• First Name: Hiền • Last Name: Lê Thị Thanh • Email: thanhhien4176@gmail.com • Telephone: 0911090921 • Address 1: 235, Nguyễn Trãi, Quận 1 • City: Hồ Chí Minh • Region / State: Ho Chi Minh City • Zip / Postal Code: 1321321 • Country: Viet Nam • Login name: thanhhien • Password: 123456 • Password Confirm: 123456 **Expected Result:** • Hệ thống xử lý dữ liệu và tạo tài khoản mới thành công. • Hiển thị thông báo: "Your Account Has Been Created!" • Chuyển hướng đến giao diện trang chủ của hệ thống • Header thay đổi: từ "Login or Register" sang "Welcome thanhhien" • Dữ liệu người dùng được ghi vào hệ thống và session đăng nhập được thiết lập.		
RE_02	**Description:** Đăng ký không thành công với email đã tồn tại trong hệ thống. **Pre-condition:** • Người dùng chưa có tài khoản.	**Test reuslt:** Passed	**Notes:** Kiểm tra khả năng xác thực

	• Login name và email sử dụng để đăng ký đã tồn tại trong hệ thống. • Truy cập được trang "Login or Register" của website https://automationteststore.com **Steps:** • Truy cập trang web: https://automationteststore.com/index.php?rt=account/login • Tại vùng "I AM A NEW CUSTOMER", chọn Continue. • Tại form Register Account, nhập đầy đủ các trường thông tin. • Tick chọn checkbox "I have read and agree to the Privacy Policy". • Chọn Continue. **Test Data:** • First Name: Hiền • Last Name: Lê Thị Thanh • Email: thanhhien4176@gmail.com • Telephone: 0911090921 • Address 1: 235, Nguyễn Trãi, Quận 1 • City: Hồ Chí Minh • Region / State: Ho Chi Minh City • Zip / Postal Code: 1321321 • Country: Viet Nam • Login name: thanhhien • Password: 123456 • Password Confirm: 123456		email đã tồn tại trong hệ thống hay chưa.

	Expected Result: • Hệ thống từ chối việc đăng ký tài khoản. • Hiển thị thông báo "Error: E-Mail Address is already registered!"		
RE_03	**Description:** Đăng ký không thành công nếu dữ liệu đầu vào thiếu một trong số các trường thông tin bắt buộc. **Pre-condition:** • Người dùng chưa có tài khoản. • Login name và email sử dụng để đăng ký chưa tồn tại trong hệ thống. • Truy cập được trang "Login or Register" của website https://automationteststore.com **Steps:** • Truy cập trang web: https://automationteststore.com/index.php?rt=account/login • Tại vùng "I AM A NEW CUSTOMER", chọn Continue • Tại form Register Account, nhập thông tin các trường dữ liệu và để trống trường "City" • Tick chọn checkbox "I have read and agree to the Privacy Policy." • Chọn Continue **Test Data:** • First Name: Hiền • Last Name: Lê Thị Thanh • Email: thanhhien4176@gmail.com	**Test reuslt:** Passed	**Note:** Kiểm tra khả năng rà soát thông tin của hệ thống

	• Telephone: 0911090921 • Address 1: 235, Nguyễn Trãi, Quận 1 • Region / State: Ho Chi Minh City • Zip / Postal Code: 1321321 • Country: Viet Nam • Login name: thanhhien • Password: 123456 • Password Confirm: 123456 **Expected Result:** • Hệ thống từ chối việc đăng ký tài khoản. • Hiển thị thông báo "City must be between 3 and 128 characters!"		
RE_04	**Description:** Đăng ký không thành khi tên đăng nhập chứa các ký tự đặc biệt như: !, @, #, $, %, ^, &, *, (,) **Pre-condition:** • Người dùng chưa có tài khoản. • Login name và email sử dụng để đăng ký chưa tồn tại trong hệ thống. • Truy cập được trang "Login or Register" của website https://automationteststore.com **Steps:** • Truy cập trang web: https://automationteststore.com/index.php?rt=account/login • Tại vùng "I AM A NEW CUSTOMER", chọn Continue • Tại form Register Account, nhập đầy đủ thông tin cho các trường dữ liệu.	**Test reuslt:** Passed	**Notes:** Kiểm tra khả năng xử lý login name chứa ký tự không hợp lệ

	• Tick chọn checkbox "I have read and agree to the Privacy Policy." • Chọn Continue. **Test Data:** • First Name: Hiền • Last Name: Lê Thị Thanh • Email: thanhhien4176@gmail.com • Telephone: 0911090921 • Address 1: 235, Nguyễn Trãi, Quận 1 • Region / State: Ho Chi Minh City • Zip / Postal Code: 1321321 • Country: Viet Nam • Login name: thanhhien@ • Password: 123456 • Password Confirm: 123456 **Expected Result:** • Hệ thống từ chối việc đăng ký tài khoản. • Hiển thị thông báo "Login name must be alphanumeric only and between 5 and 64 characters!"		
RE_05	**Description:** Hệ thống từ chối tạo tài khoản khi kiểm tra thấy "Telephone" – trường thông tin không bắt buộc có định dạng dữ liệu không đúng với logic (Số điện thoại gồm 12 số). **Pre-condition:** • Người dùng chưa có tài khoản. • Login name và email sử dụng để đăng ký chưa tồn tại trong hệ thống.	**Test reuslt:** Failed	**Notes:** Kiểm tra khả năng xác thực tính logic của dữ liệu của

	• Truy cập được trang "Login or Register" của website https://automationteststore.com • Dữ liệu của các trường thông tin bắt buộc hợp lệ. **Steps:** • Truy cập trang web: https://automationteststore.com/index.php?rt=account/login • Tại vùng "I AM A NEW CUSTOMER", chọn Continue. • Tại form Register Account, nhập thông tin các trường dữ liệu theo đúng yêu cầu. • Nhập "Telephone" gồm 12 số. • Tick chọn checkbox "I have read and agree to the Privacy Policy". • Chọn Continue. **Test Data:** • First Name: Hiền • Last Name: Lê Thị Thanh • Email: thanhhien4176@gmail.com • Telephone: 091109092178 • Address 1: 235, Nguyễn Trãi, Quận 1 • Region / State: Ho Chi Minh City • Zip / Postal Code: 1321321 • Country: Viet Nam • Login name: thanhhien • Password: 123456 • Password Confirm: 123456		trường thông tin không bắt buộc.

	Expected Result: • Hệ thống từ chối việc đăng ký tài khoản. • Hiển thị thông báo lỗi dữ liệu không hợp lệ.		

Bảng 3. 2: Danh sách Testcase chức năng Login

TC ID	**Function: Login**		
LG_01	**Description:** Đăng nhập thành công với Login name và Password hợp lệ. **Pre-condition:** • Tài khoản đã tồn tại trong hệ thống với trạng thái đang hoạt động. • Người dùng truy cập được vào trang đăng nhập của hệ thống. **Steps:** • Truy cập trang: https://automationteststore.com/index.php?rt=account/login • Nhập login name và password hợp lệ • Chọn Login **Test Data:** • Login name: thanhhien • Password: 123456 **Expected Result:** • Chuyển hướng đến trang chủ của hệ thống với tài khoản cá nhân hiện hành là Hiền • Header hiển thị: “Welcome Hiền!”	**Test result:** Passed	**Notes:** Kiểm tra khả năng xác thực thông tin đăng nhập của hệ thống

LG_02	**Description:** Đăng nhập không thành công với "Login Name" không hợp lệ. **Pre-condition:** • Tài khoản đã tồn tại trong hệ thống với trạng thái đang hoạt động. • Người dùng truy cập được vào trang đăng nhập của hệ thống. **Steps:** • Truy cập trang: https://automationteststore.com/index.php?rt=account/login • Nhập "Login Name" và "Password" hợp lệ • Chọn Login. **Test Data:** • Login name: thanhhien123 • Password: 123456 **Expected Result:** • Hệ thống từ chối việc đăng nhập. • Hiển thị thông báo "Error: Incorrect login or password provided."	**Test result:** Passed	**Notes:** Kiểm tra khả năng xác thực dữ liệu đăng nhập có hợp lệ hay không.

3.1.2. Quản lý mua sản phẩm Haircare

Bảng 3. 3: Danh sách Testcase chức năng Show products

TC ID	Function: Show products		
SP_01	**Description:** Hệ thống hiển thị các sản phẩm trong danh mục Haircare (gồm các sản phẩm trong danh mục Shampoo và danh mục Conditioner) với số lượng	**Test reuslt:**	**Note**

	phù hợp với tùy chọn "Per Page" (Per page là số lượng sản phẩm được phép hiển thị trên mỗi trang). **Pre-condition:** • Tổng số lượng sản phẩm trong danh mục Shampoo và Conditioner <= Số lượng của tùy chọn "Per Page" **Steps:** • Truy cập trang web: https://automationteststore.com/index.php?rt=account/login • Đăng nhập thành công vào hệ thống • Chọn danh mục Haircare • Vào danh mục Conditioner. • Quan sát số lượng sản phẩm có trong danh mục Conditioner. • Vào danh mục Shampoo. • Quan sát số lượng sản phẩm có trong danh mục Shampoo. • Quay trở lại danh mục Haircare. • Chọn "Per page" = 20. • Quan sát số lượng các sản phẩm hiển thị trong danh mục Haircare. **Test Data:** • Per page = 20. **Expected Result:** • Các sản phẩm hiển thị trong danh mục Haircare phải bao gồm các sản phẩm có trong danh mục Shampoo và Conditioner.	Failed	

Bảng 3. 4: Danh sách Testcase chức năng Review

TC ID	Function: Review		
RV_01	**Description:** Không ghi nhận bài đánh giá nếu người dùng viết đánh giá bằng mã HTML. **Pre-condition:** • Tài khoản đã tồn tại trong hệ thống với trạng thái đang hoạt động. • Người dùng truy cập được vào trang của hệ thống. **Steps:** • Truy cập trang web: https://automationteststore.com/index.php?rt=account/login • Đăng nhập thành công vào hệ thống. • Chọn danh mục Haircare. • Chọn vào sản phẩm Seaweed Conditioner. • Chọn mục "Review" • Điền đầy đủ thông tin. • Chọn Submit. **Test Data:** • Rating: 4 sao • Your name: Lê Thị Thanh Hiền • Your Review: "<!DOCTYPE html> <html> <body>	**Test result:** Failed	**Notes:** Kiểm tra khả năng xác thực tính hợp lệ của dữ liệu của hệ thống.

	 Eau Parfumee Au The Vert Shampoo </body> </html>” • Code: e0492a **Expected Result:** • Không ghi nhận bài đánh giá. • Hiển thị thông báo lỗi.		

Bảng 3. 5: Danh sách Testcase chức năng Add to cart

TC ID	Function: Add to cart		
AC_01	**Description:** Không cho phép thêm sản phẩm vào giỏ hàng khi sản phẩm đã hết hàng, ngay cả khi người dùng sử dụng phím Enter từ bàn phím. **Pre-condition:** • Sản phẩm đang ở trạng thái Out of Stock. • Nút “Add to Cart” đã được đổi thành “Out of Stock” và đã bị vô hiệu hóa. • Phím Enter trên bàn phím bị vô hiệu hóa. **Steps:** • Truy cập trang web: https://automationteststore.com/index.php?rt=account/login • Đăng nhập thành công vào hệ thống.	**Test reuslt:** Failed	**Notes:**

	• Chọn danh mục Haircare. • Chọn vào sản phẩm Eau Parfumee au The Vert Shampoo. • Tại trường "Qty" nhập số lượng là 2. • Ấn Enter trên bàn phím. **Test Data:** • Qty = 2 **Expected Result:** • Hệ thống không ghi nhận việc thêm sản phẩm vào giỏ hàng. • Hiển thị thông báo sản phẩm đã hết hàng không thể thêm vào giỏ hàng.		
AC_02	**Description:** Thêm sản phẩm vào giỏ hàng thành công. **Pre-condition:** • Sản phẩm còn hàng. • Nút "Add to cart" không bị vô hiệu hóa. **Steps:** • Truy cập trang web: https://automationteststore.com/index.php?rt=account/login • Đăng nhập thành công vào hệ thống. • Chọn danh mục Haircare. • Chọn vào sản phẩm Seaweed Conditioner. • Chọn "Add to cart". **Test Data:** **Expected Result:**	**Test reuslt:** Passed	**Notes:**

	• Sản phẩm được thêm vào giỏ hàng thành công. • Hệ thống chuyển hướng đến trang giỏ hàng để người dùng kiểm tra.		

Bảng 3. 6: Danh sách Testcase Checkout

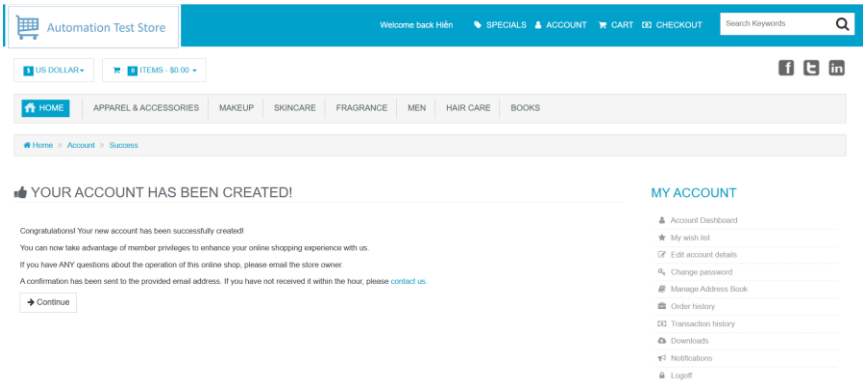
TC ID	**Function: Checkout**		
PO_01	**Description:** Hệ thống cho phép đặt hàng thành công các sản phẩm có trong giỏ hàng. **Pre-condition:** • Các sản phẩm trong giỏ hàng đều còn hàng. • Thông tin đặt hàng được lấy từ thông tin dùng để đăng ký tài khoản. **Steps:** • Truy cập trang web: https://automationteststore.com/index.php?rt=account/login • Đăng nhập thành công vào hệ thống. • Chọn danh mục Haircare. • Chọn vào sản phẩm Seaweed Conditioner. • Chọn "Add to cart". • Tại trang giỏ hàng, chọn "Checkout". • Tại trang Payment, kiểm tra thông tin giao hàng. • Chọn "Confirm Order" để tiến hành đặt hàng. **Test Data:** • Sản phẩm trong giỏ hàng. • Thông tin đặt hàng.	**Test reuslt:** Passed	**Notes:**

	Expected Result: • Hệ thống ghi nhận việc đặt hàng. • Hiển thị thông báo đặt hàng thành công "Your Order Has Been Processed!" • Hiển thị mã đặt hàng. • Hiển thị liên kết để xem "invoice" • Hiển thị lời cảm ơn.		

3.2. Danh sách Bug Report

Bảng 3. 7: Bug report chức năng Register

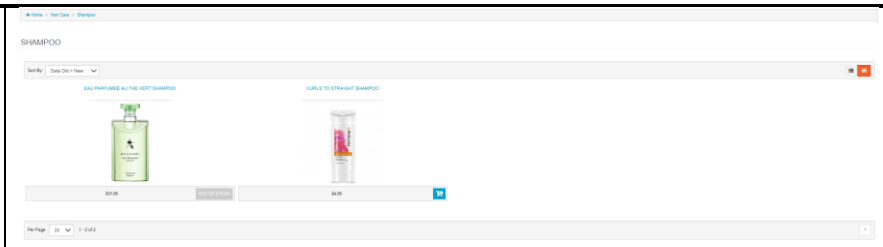
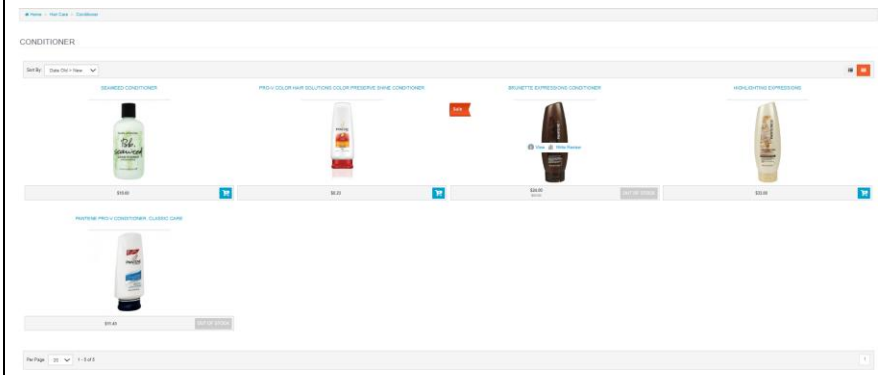
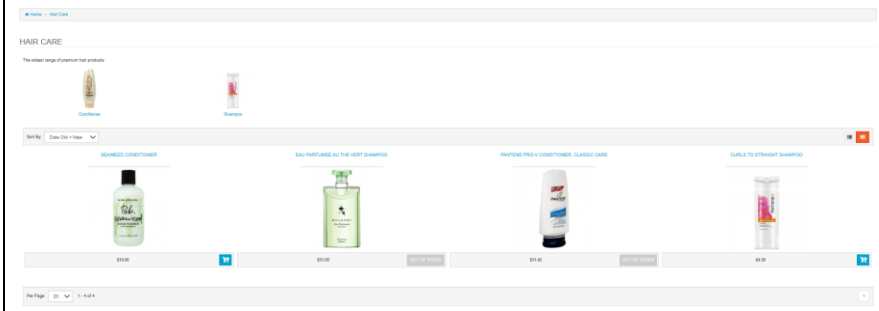
Bug ID	Function: Register	Note
RE_05	**Summary:** Nhập thông tin cho trường "Telephone" với dữ liệu không hợp lệ (Số điện thoại gồm 12 số) hệ thống vẫn chấp nhận. **Steps to reproduce (expected & observed)** • Truy cập trang web: https://automationteststore.com/index.php?rt=account/login • Tại vùng "I AM A NEW CUSTOMER", chọn Continue. • Tại form Register Account, nhập thông tin các trường dữ liệu theo đúng yêu cầu. • Nhập "Telephone" gồm 12 số. • Tick chọn checkbox "I have read and agree to the Privacy Policy". • Chọn Continue. **Expected Result:** • Hệ thống từ chối việc đăng ký tài khoản.	

	• Hiển thị thông báo lỗi dữ liệu không hợp lệ. **Observed result:** Hệ thống chấp nhận và đăng ký thành công. **Date found:** 05/08/2025 **Reproducibility:** Always **Reported By:** Thanh Hiền **Review By:** Thanh Hiền **Bug screen:** Telephone: 091109092178 *Hình 3. 1: Dữ liệu đầu vào của Bug ID RE_05*  *Hình 3. 2: Bug screen của Bug ID RE_05*	

Bảng 3. 8: Bug report chức năng Show products

Bug ID	Function: Show products	Note
SP_01	**Summary:** Tổng số lượng sản phẩm trong danh mục Shampoo và Conditioner <= Số lượng của tùy chọn "Per Page" nhưng tại trang Haircare lại hiển thị số lượng sản phẩm ít hơn. **Steps to reproduce (expected & observed)**	

	• Truy cập trang web: https://automationteststore.com/index.php?rt=account/login • Đăng nhập thành công vào hệ thống • Chọn danh mục Haircare • Vào danh mục Conditioner. • Quan sát số lượng sản phẩm có trong danh mục Conditioner. • Vào danh mục Shampoo. • Quan sát số lượng sản phẩm có trong danh mục Shampoo. • Quay trở lại danh mục Haircare. • Chọn "Per page" = 20. • Quan sát số lượng các sản phẩm hiển thị trong danh mục Haircare. **Expected Result:** Các sản phẩm hiển thị trong danh mục Haircare phải bao gồm các sản phẩm có trong danh mục Shampoo và Conditioner. **Observed result:** Tại danh mục Haircare trang web chỉ hiển thị một vài sản phẩm của danh mục Shampoo và Conditioner không hiển thị tất cả. **Date found:** 03/08/2025 **Reproducibility:** Always **Reported By:** Thanh Hiền **Review By:** Thanh Hiền **Bug screen:**	

	 *Hình 3. 3: Các sản phẩm trong danh mục Shampoo*  *Hình 3. 4: Các sản phẩm trong danh mục Conditioner*  *Hình 3. 5: Bug screen của Bug ID SP_01*	

Bảng 3. 9: Bug report chức năng Review

Bug ID	Function: Review	Note
RV_01	**Summary:** Hệ thống ghi nhận bài đánh giá khi người dùng viết đánh giá bằng mã HTML. **Steps to reproduce (expected & observed)** • Truy cập trang web: https://automationteststore.com/index.php?rt=account/login	

- Đăng nhập thành công vào hệ thống.
- Chọn danh mục Haircare.
- Chọn vào sản phẩm Seaweed Conditioner.
- Chọn mục “Review”
- Điền đầy đủ thông tin.
- Chọn Submit.

Expected Result:

- Không ghi nhận bài đánh giá.
- Hiển thị thông báo lỗi.

Observed result: Trang web vẫn chấp nhận bài đánh giá và hiển thị thông báo thành công

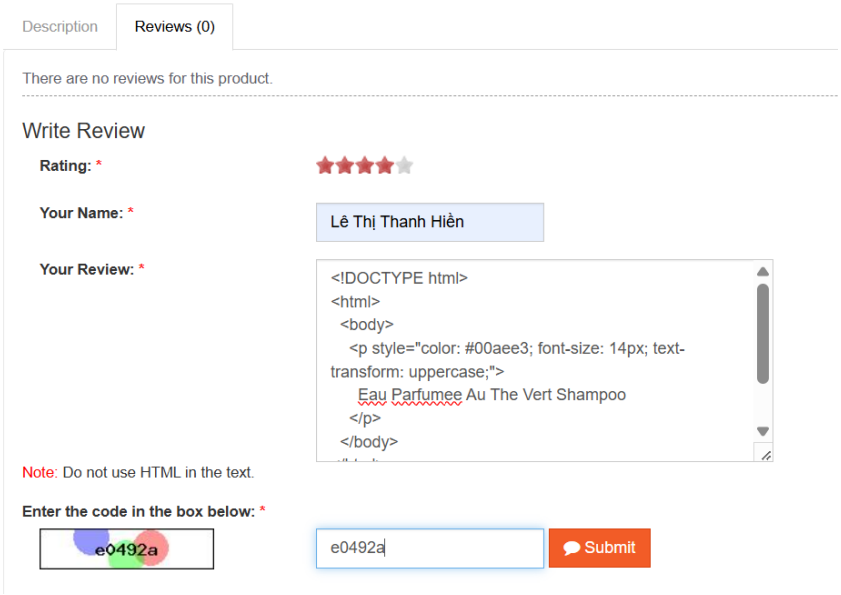
Date found: 03/08/2025

Reproducibility: Always

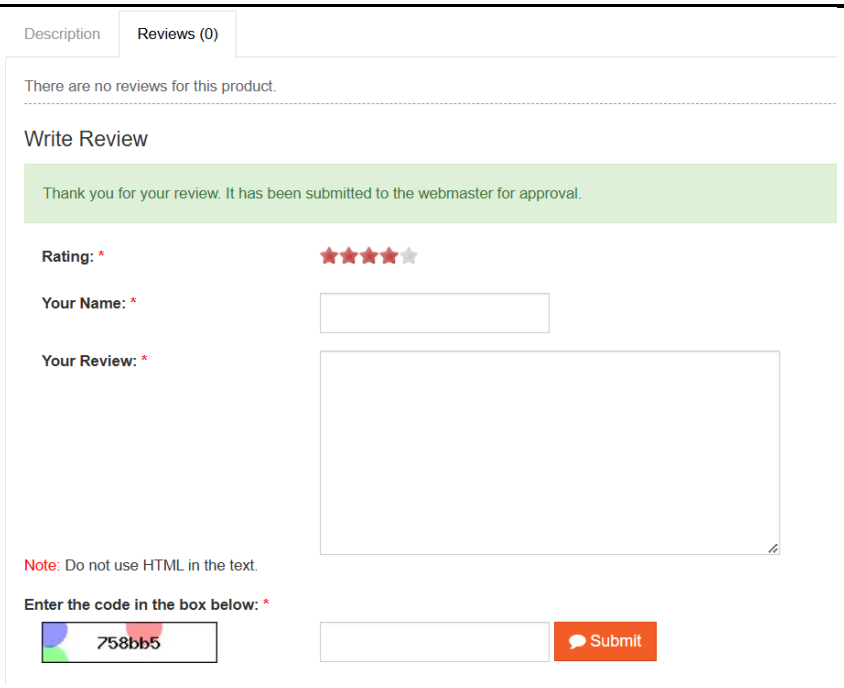
Reported By: Thanh Hiền

Review By: Thanh Hiền

Bug screen:



Hình 3. 6: Dữ liệu đầu vào của Bug ID RV_01

	 *Hình 3. 7: Bug screen của Bug ID RV_01*	

Bảng 3. 10: Bug report chức năng Add to cart

Bug ID	Function: Add to cart	Note
AC_01	**Summary:** Hệ thống cho phép thêm sản phẩm vào giỏ hàng ngay cả khi sản phẩm đã hết hàng khi người dùng thay đổi số lượng sản phẩm muốn mua và ấn Enter từ bàn phím. **Steps to reproduce (expected & observed)** • Truy cập trang web: https://automationteststore.com/index.php?rt=account/login • Đăng nhập thành công vào hệ thống. • Chọn danh mục Haircare. • Chọn vào sản phẩm Eau Parfumee au The Vert Shampoo. • Tại trường "Qty" nhập số lượng là 2. • Ấn Enter trên bàn phím.	

Expected Result:

- Hệ thống không ghi nhận việc thêm sản phẩm vào giỏ hàng.
- Hiển thị thông báo sản phẩm đã hết hàng không thể thêm vào giỏ hàng.

Observed result: Hệ thống thêm sản phẩm vào giỏ hàng ở trạng thái không còn hàng.

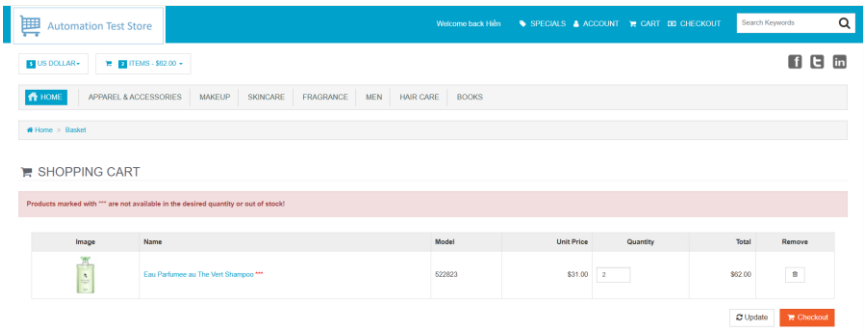
Date found: 03/08/2025

Reproducibility: Always

Reported By: Thanh Hiền

Review By: Thanh Hiền

Bug screen:



Hình 3. 8: Bug screen của Bug ID AC_01

CHƯƠNG 4: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1. Kết quả đạt được

- Hoàn thiện bộ test case cho các chức năng trọng tâm bao gồm: quản lý tài khoản, duyệt danh mục sản phẩm và quy trình mua sản phẩm.
- Thực hiện kiểm thử chức năng (Functional Testing), kiểm thử giao diện (UI Testing) và kiểm thử luồng người dùng (User Flow Testing), đảm bảo đánh giá được cả tính đúng đắn và tính thân thiện của hệ thống.
- Phát hiện, ghi nhận và phân tích các lỗi hoặc hành vi không mong muốn để phục vụ cải thiện chất lượng hệ thống.
- Xác nhận rằng phần lớn các chức năng cốt lõi hoạt động ổn định, đáp ứng yêu cầu nghiệp vụ và mang lại trải nghiệm người dùng tương đối tốt.

4.2. Hạn chế và đề xuất cải thiện

4.2.1. Hạn chế

- Phạm vi kiểm thử mới chỉ tập trung vào các chức năng chính của một danh mục cụ thể, chưa bao quát toàn bộ các danh mục khác của trang web và các tính năng khác của hệ thống.
- Chưa thực hiện kiểm thử phi chức năng (Non-functional Testing) như: hiệu năng, khả năng chịu tải hoặc bảo mật.
- Chưa kiểm tra đầy đủ các trường hợp biên (edge cases) và tình huống nhập liệu không hợp lệ phức tạp.
- Thời gian kiểm thử có hạn nên chưa tiến hành được kiểm thử trên nhiều môi trường trình duyệt và thiết bị khác nhau.

4.2.2. Đề xuất

- Mở rộng phạm vi kiểm thử để bao quát toàn bộ chức năng của hệ thống, bao gồm cả các tính năng nâng cao và tùy chọn mở rộng.
- Bổ sung kiểm thử phi chức năng, đặc biệt là kiểm thử hiệu năng (Performance Testing) và bảo mật (Security Testing).
- Tăng cường kiểm thử trên nhiều nền tảng (trình duyệt, hệ điều hành, thiết bị di động) để đảm bảo tính tương thích.

- Xây dựng kịch bản kiểm thử tự động (Automation Testing) cho các chức năng lặp lại nhằm tiết kiệm thời gian và giảm sai sót thủ công.

4.3. Hướng phát triển tiếp theo

- Phát triển và áp dụng bộ kiểm thử tự động để hỗ trợ kiểm thử hồi quy (Regression Testing) khi hệ thống có cập nhật.
- Tích hợp các công cụ giám sát hiệu năng (Monitoring Tools) để đánh giá và cải thiện tốc độ phản hồi.
- Triển khai quy trình kiểm thử liên tục (Continuous Testing) trong mô hình CI/CD để đảm bảo chất lượng phần mềm xuyên suốt vòng đời phát triển.
- Nghiên cứu và áp dụng các kỹ thuật kiểm thử nâng cao như A/B Testing để tối ưu trải nghiệm người dùng.
- Định kỳ đánh giá, bảo trì và cập nhật kịch bản kiểm thử để phù hợp với những thay đổi của hệ thống.

TÀI LIỆU THAM KHẢO

Sách

1. Cường, T. B., & Mận, N. Đ. (2011). *Kiểm thử và đảm bảo chất lượng phần mềm*. Nhà xuất bản Bách Khoa Hà Nội.
2. (n.d.). *Giáo trình thực hành kiểm thử phần mềm*. FAHASA.
3. Patton, R. (2005). *Software testing* (2nd ed.). Sams Publishing.

Website

1. SmartBear. (n.d.). *Software testing methodologies*. SmartBear. https://smartbear.com/learn/automated-testing/software-testing-methodologies/
2. Wikipedia. (2025, February 6). *Continuous testing*. In *Wikipedia*. https://en.wikipedia.org/wiki/Continuous_testing
3. Wikipedia. (2025, March 2). *Test automation*. In *Wikipedia*. https://en.wikipedia.org/wiki/Test_automation
4. SmartBear. (n.d.). *Agile test automation*. SmartBear. https://smartbear.com/learn/automated-testing/testing-in-agile-environments/
5. VTI Academy. (n.d.). *Automation test là gì? Những điều bạn nên biết về nghề này*. VTI Academy. https://vtiacademy.edu.vn/automation-test-la-gi.html